

Bericht

Alisa Pesteritz, Saliha Altan, Wiktoria Cholewa,
Dominik Falk, Andreas Kasarinow, Johanna Kleidorfer,
Adrika Narasimhan, Rosina Röckseisen

17. Juli 2026

1 Einleitung

Die Analyse von Genexpressionsdaten spielt eine zentrale Rolle in der Bioinformatik und Molekularbiologie. Durch die Untersuchung von Expressionsmustern lassen sich Zusammenhänge zwischen Genen identifizieren, biologische Prozesse besser verstehen und potenzielle Biomarker für Krankheiten erkennen. Genexpressionsdatensätze enthalten häufig eine große Anzahl an Genen, was eine effiziente Klassifizierung und Visualisierung der Daten erheblich erschwert.

Im Rahmen dieses Projekts wurde eine Softwareanwendung entwickelt, die den Import von Genexpressionsdaten ermöglicht und diese mithilfe einer hierarchischen Clusteranalyse auswertet. Ziel der Anwendung ist es, ähnliche Gene beziehungsweise Patienten anhand ihrer Expressionsprofile zu gruppieren und die Ergebnisse übersichtlich darzustellen. Die Visualisierung erfolgt in einem interaktiven Dashboard, das sowohl die Dendrogramme als auch eine Heatmap umfasst. Während die Dendrogramme die hierarchischen Beziehungen zwischen den Clustern veranschaulichen, bietet die Heatmap einen intuitiven Überblick über die Expressionswerte und deren Muster.

Die Entwicklung der Anwendung erfolgte im Rahmen einer Gruppenarbeit. Dabei übernahmen die einzelnen Teammitglieder unterschiedliche Aufgabenbereiche. Die folgende Tabelle zeigt die Aufgabenbereiche sowie die jeweils dafür zuständigen Personen.

Aufgabenbereich	Zuständig
Pathways und Einlesen der Daten	Alisa
Datenvorbereitung und Normalisierung	Rosina
Distanzfunktionen und Linkage-Verfahren	Wiktorja
Dendrogramme	Dominik
Heatmap	Saliha
Graphische Oberfläche und Layout-Design	Adrika
Serverlogik	Andreas
Farbgebung und biologisch-medizinische Grundlagen	Johanna

Eine genauere Erläuterung der Aufgabenbereiche erfolgt in den folgenden Kapiteln. Die vorliegende Projektarbeit beschreibt sowohl die theoretischen Grundlagen als auch die technische Umsetzung der entwickelten Anwendung.

2 Grundlagen

2.1 Biologische Grundlagen

2.1.1 Genexpression und -regulation

Die im Folgenden analysierten Genexpressionsdaten beschreiben die Aktivität eines Gens und damit, wann und in welcher Intensität ein Gen abgelesen wird. Dies entscheidet über den Phänotyp einer Zelle, denn auch wenn die Zellen eines Organismus dieselbe genetische Information in Form von DNA enthalten, erfüllen sie doch sehr unterschiedliche Funktionen. Die Genexpression folgt dabei dem Zentralen Dogma der Molekularbiologie: bei der Transkription wird die in der DNA kodierte Information in RNA und anschließend mittels der Translation in Proteine überführt [27].

Dieser Prozess der Genexpression unterliegt einer zellinternen Genregulation. Dabei binden Transkriptionsfaktoren an regulatorische DNA-Abschnitte wie Promotoren und Enhancer und rekrutieren darüber die RNA-Polymerase, welche das Ablesen des jeweiligen Genabschnitts startet. Je nach gebundenem Transkriptionsfaktor wird die Transkription gehemmt oder aktiviert, es kommt also entweder zum Ablesen oder nicht. Transkriptionsfaktoren sind dabei selbst Proteine, die als Genprodukte anderer Gene entstehen. Je nach Zelltyp, Entwicklungsstadium und äußeren Signalen sind verschiedene Transkriptionsfaktoren vorhanden und aktiv. Diese kaskadierenden Genregulationsnetzwerke erklären, warum trotz identischer DNA in jeder Zelle andere Gene ausgelesen werden [27].

Die Feinjustierung der Genexpression umfasst mehrere nachgelagerte Ebenen. Post-transkriptionell wird beispielsweise durch microRNAs oder alternatives Spleißen reguliert, diese Prozesse entscheiden welche und wie viel mRNA für die Translation zur Verfügung steht. Translational wird die Proteinbiosyntheserate kontrolliert, welche bestimmt, wie viele funktionale Proteine aus einer gegebenen mRNA-Menge tatsächlich entstehen. In der post-translationalen Ebene findet schließlich die Proteinmodifikation (z.B. Phosphorylierung, Ubiquitinierung) statt. Eine Abnormalität in einer dieser Kontrollebenen kann zur Fehlregulation der Genexpression und infolgedessen zu Krankheitsbildern führen [27].

Die Datenerhebung der möglicherweise dafür verantwortlichen RNA-Mengen erfolgt nach heutigem Stand der Technik meist über das High-Throughput-Verfahren des Next-Generation-Sequencing (RNA-Seq). In Ausnahmefällen werden noch ältere Microarray-Technologien verwendet, diese haben allerdings eine geringere Sensitivität und einen eingeschränkteren dynamischen Messbereich als RNA-Seq [44]. Der bioinformatische Workflow führt dabei von der RNA-Isolation über das Alignment an ein Referenzgenom bis hin zur Generierung einer rohen Count-Matrix [24]. Diese Matrix bildet das Ergebnis der Sequenzierung ab, sie enthält für jedes Gen (Zeilen) und jede untersuchte Probe oder jeden Patienten (Spalten) die Anzahl der Reads. Ein solches tabellarische Format erwartet auch der CSV-Upload des in diesem Projekt entwickelten Dashboards (siehe Kap. 6).

Bei diesen Rohdaten kommt es jedoch unweigerlich zu systematischen Verzerrungen, bedingt durch variierende Genlängen und stark schwankende Sequenziertiefen zwischen den einzelnen Probenläufen. Die Sequenziertiefe bezeichnet dabei die Gesamtzahl der in einem Lauf erzeugten Reads für eine Probe und kann zwischen den Proben stark variieren. Ein direkter Vergleich zweier Proben unterschiedlicher Sequenziertiefen würde also fälschlicherweise einen Unterschied in der Genaktivität suggerieren, obwohl dieser lediglich an der Datenmenge liegt. Für eine valide biologische Vergleichbarkeit müssen die Datensätze daher zwingend mathematisch normalisiert werden, bevor sie miteinander verglichen werden können (siehe Kap. 4.4) [30].

Die Analyse der Genexpressionsdaten ist vorrangig für die Onkologie von großem Interesse. Dabei wird die Genaktivität quantifiziert und analysiert, wofür das Transkriptom, also die Gesamtheit aller zu einem bestimmten Zeitpunkt in einer Zelle transkribierten RNA-Moleküle, betrachtet werden muss [7]. Denn nicht nur das Genom, also die strukturelle DNA und deren Mutationen, ist entscheidend für die Entstehung und das Fortschreiten vieler Krankheitsbilder, sondern maßgeblich auch die zelluläre Aktivität. So lassen sich beispielsweise pathophysiologische Prozesse wie unkontrolliertes Zellwachstum oder die Metastasierung bei strukturell intakten Genen beobachten, wenn diese fehlerhaft reguliert (über- oder unterexprimiert) sind [42].

Zu berücksichtigen ist dabei außerdem, dass Genexpression kein synchroner, sondern ein zeitlich gestaffelter, asynchroner Prozess ist. Wird eine Zelle beispielsweise durch einen Wachstumsfaktor oder eine DNA-Schädigung stimuliert, werden nicht alle beteiligten Gene gleichzeitig abgelesen. Zuerst reagieren einzelne, meist regulatorisch wirkende Gene innerhalb weniger Minuten, deren Genprodukte wiederum weitere, nachgeschaltete Zielgene erst zeitversetzt aktivieren [27]. Eine einzelne RNA-Seq-Messung bildet dabei stets nur eine Momentaufnahme dieses dynamischen Prozesses ab. Zwei Gene, die demselben biologischen Signalweg angehören, müssen in einer solchen Momentaufnahme also nicht zwangsläufig ein gleich starkes Expressionsniveau aufweisen.

2.1.2 Pathways und die KEGG-Datenbank

Da viele biologische Prozesse jedoch nicht monogen ablaufen, reicht die Betrachtung einzelner Gene meist nicht für die biologische Interpretierbarkeit. Deshalb müssen funktionell zusammenhängende Gene gemeinsam betrachtet werden, um Muster und Zusammenhänge erkennen zu können. Dementsprechend ist es wichtig die Gene zusammen darzustellen, welche an denselben biologischen Signalwegen beteiligt sind, dies kann durch sogenannte Pathways realisiert werden. Dabei handelt es sich um kuratierte Netzwerke von Genen bzw. Genprodukten, die gemeinsam einen gewissen Signalweg abbilden, wie beispielsweise den einer Signaltransduktion. Die Pathways reduzieren für die Analyse der Daten die Komplexität und erhöhen dabei gleichzeitig die Interpretierbarkeit der Ergebnisse, da sie den funktionalen Kontext liefern [22].

Eine der wichtigsten Referenzdatenbanken, welche hunderte dieser kuratierten Pathways für unterschiedlichste Organismen bereithält, ist die Kyoto Encyclopedia of Genes and Genomes (KEGG) [20]. KEGG bildet Signalwege, Stoffwechselwege und krankheitsassoziierte Prozesse als standardisierte grafische annotierte Netzwerke ab und beinhaltet pro Pathway die zusammengehörigen Gene. Damit können die Genexpressionsdaten direkt auf die bekannten biologischen Zusammenhänge abgebildet werden.

Das in diesem Projekt entwickelte Dashboard erlaubt es den Nutzern verschiedenste Pathways der KEGG-Datenbank auszuwählen. Damit lässt sich die Analyse gezielt auf die für einen bestimmten Signalweg relevanten Gene eingrenzen, anstatt anstatt alle in der Count-Matrix enthaltenen Gene gemeinsam betrachten zu müssen. Genauer zur technischen Implementierung ist in Kapitel 3 zu finden.

2.2 Medizinische Grundlagen

2.2.1 Klinische Relevanz der Genexpressionanalyse

Das entwickelte Tumorboard-Dashboard findet vorrangig im klinischen Kontext, bei der Entwicklung personalisierter Therapiepläne, Anwendung. Ärzte sind hierbei besonders an der Genexpression und deren Regulation interessiert, da herkömmliche histopathologische, also rein visuelle Untersuchungen des Tumorgewebes die molekulare Heterogenität von Tumoren meist nicht vollständig abbilden können [42]. Denn zwei histologisch identische Tumore können auf molekularer Ebene vollkommen unterschiedliche Aktivitätsprofile aufweisen und entsprechend

unterschiedlich auf verschiedene Therapien ansprechen. Diese Informationsebene kommt im klinischen Bereich besonders für die Diagnosen, die Prognoseabschätzung und die Vorhersage der Erfolgchancen bei Therapieansätzen zum Einsatz [29]. Beispiele aus der heutigen Zeit für den Einsatz der Genexpressionsdaten sind die Bestimmung des Ursprungsgewebes bei Tumoren unbekannter Herkunft, die Therapieentscheidung bei Brustkrebs sowie die Einschätzung des Rezidivrisikos, also der Wahrscheinlichkeit, dass die Erkrankung nach erfolgreicher Genesung erneut auftritt [29].

2.2.2 Das Molekulare Tumorboard

Das Molekulare Tumorboard (MTB) ist eine interdisziplinäre Expertenkonferenz, in der für besondere Krankheitsbilder oder spezielle Verläufe individualisierte Therapiepläne entwickelt werden. Hier dienen die genomischen und transkriptomischen Profile der Entscheidungsfindung [39].

2.2.3 Das Dashboard als Clinical Decision Support System

Das Dashboard fungiert hierbei als interaktives Clinical Decision Support System (CDSS) [39]. Es bietet mit der Heatmap und den Dendrogrammen eine gemeinsame, leicht verständliche, visuell dargestellte Diskussionsgrundlage für die beteiligten Experten. Es erlaubt einem interdisziplinären Kollektiv aus Onkologen, Pathologen, Chirurgen und Bioinformatikern die medizinischen Analysen ohne Programmierkenntnisse in Echtzeit über eine grafische Benutzeroberfläche zu steuern und anschließend gemeinsam darüber zu beraten.

Das Ziel des interdisziplinären Gremiums besteht dementsprechend darin, die genomischen und transkriptomischen Profile der Patienten mithilfe des CDSS gemeinschaftlich zu evaluieren und mit den daraus generierten Informationen eine individuelle Therapieempfehlung auszuarbeiten. Denn wichtige und weitreichende Therapieentscheidungen können nicht von einem einzelnen Arzt isoliert verantwortet werden [28]. Studien belegen, dass diese Art der computergestützten Therapiewahl die Behandlungsergebnisse maßgeblich verbessern kann [18]. Durch die Anpassungsmöglichkeiten, wie die Auswahl spezifischer biologischer Pathways, die Modifikation der Normalisierungsverfahren, die Wahl des Cluster-Algorithmus (z. B. Single, Average und Complete Linkage) und der farblichen Darstellung kann das Dashboard zum zentralen Visualisierungsmedium in der Konferenz werden. Das ermöglicht den Ärzten, Hypothesen direkt während der Sitzung gemeinsam zu prüfen, molekulare Muster im Kollektiv zu interpretieren und so die bestmögliche personalisierte Therapieempfehlung für die Patienten auszuarbeiten [31].

2.2.4 Informatische Grundlagen

Die Analyse von Genexpressionsdaten basiert auf Methoden der Datenanalyse und des maschinellen Lernens, die dazu dienen, große und komplexe Datensätze systematisch auszuwerten. Da Genexpressionsdatensätze häufig eine hohe Anzahl an Genen und Proben enthalten, ist eine strukturierte Analyse erforderlich, um relevante Muster und Zusammenhänge zu erkennen. Zu den grundlegenden Verfahren zählen die explorative Datenanalyse (Exploratory Data Analysis, EDA) sowie Clustering-Methoden, die auch die Basis der in dieser Arbeit entwickelten Anwendung bilden. Die explorative Datenanalyse stellt einen der ersten Schritte nach der Datenaufbereitung dar. Ihr Ziel ist es, die Eigenschaften eines Datensatzes zu untersuchen und ein grundlegendes Verständnis für dessen Struktur zu gewinnen. Dabei werden die Daten zunächst ohne vorher festgelegte Hypothesen betrachtet. Mithilfe statistischer Kennzahlen sowie grafischer Darstellungen können Verteilungen, Zusammenhänge, Ausreißer oder andere Auffälligkeiten identifiziert werden. Die explorative Datenanalyse unterstützt somit die Beurteilung der Datenqualität und liefert wichtige Erkenntnisse für die Auswahl geeigneter Analysemethoden. Darüber hinaus können erste Hypothesen über mögliche Beziehungen innerhalb der Daten entwickelt werden [23].

Ein weiteres zentrales Verfahren ist das Clustering. Es gehört zum Bereich des unüberwachten maschinellen Lernens und dient dazu, Datenobjekte anhand ihrer Ähnlichkeit automatisch zu Gruppen, sogenannten Clustern, zusammenzufassen. Im Gegensatz zur Klassifikation liegen beim Clustering keine vorgegebenen Klassen oder Kategorien vor. Ziel ist es stattdessen, bislang unbekannte Strukturen innerhalb der Daten zu identifizieren. Dabei sollen die Elemente eines Clusters möglichst ähnliche Merkmale aufweisen, während sich verschiedene Cluster möglichst deutlich voneinander unterscheiden [33].

Im Kontext der Genexpressionsanalyse ermöglicht Clustering die Identifikation von Genen oder biologischen Proben mit ähnlichen Expressionsmustern. Die so gewonnenen Gruppen können anschließend mithilfe geeigneter Visualisierungstechniken dargestellt und interpretiert werden. Die Kombination aus explorativer Datenanalyse, Clustering und Visualisierung bildet die Grundlage für die in dieser Arbeit entwickelte Anwendung zur Analyse von Genexpressionsdatensätzen.

3 Pathways und Dateneinlesen

Die Tabelle zeigt, wie groß der Arbeitsaufwand pro Unteraufgabe ausgefallen ist.

Aufgabe	Zuständig	Arbeitsaufwand
Recherche zu Pathways und Extraktion der Daten von KEGG	Alisa	5 Stunden
Datenbank entwerfen und aufsetzen	Alisa	7 Stunden
Funktionen: Prozessierung, Datenbankabfragen und Integration	Alisa	17 Stunden
Datensätze bearbeiten	Alisa	5 Stunden
Besprechungstermine	Alle	19,5 Stunden
Berichterstellung	Alisa	5 Stunden

Der Bereich Pathways und Dateneinlesen untergliedert sich in mehrere Teilbereiche, welche im Folgenden näher erläutert werden.

3.1 Datenbeschaffung

Das zentrale Ziel des Aufgabenbereiches war die semantische Datenintegration biologischer Genexpressionsdatensätze. Konkret sollte es möglich sein, aus einem größeren Genexpressionsdatensatz, gezielt die Gene herauszufiltern, die den vom Nutzer ausgewählten biologischen Pathways zugeordnet sind. Voraussetzung dafür ist eine strukturelle Grundlage für die Verknüpfung der Pathways mit ihren zugehörigen Genen. Dafür wurde die KEGG-Datenbank verwendet, welche im Abschnitt 2.1 zusammen mit der Funktion der Pathways näher erläutert wird. KEGG stellt eine öffentliche REST-API-Schnittstelle bereit, über die die folgende Informationen abgerufen und als CSV-Dateien gespeichert wurden:

- Pathway-Liste: Alle menschlichen Pathways mit ihrer KEGG ID und ihrem Namen. Abgerufen über <https://rest.kegg.jp/list/pathway/hsa>.
- Gen-Pathway Verknüpfungen: Die Zuordnung von Genen zu ihren entsprechenden Pathways. Abgerufen über <https://rest.kegg.jp/link/pathway/hsa>.

Die CSV-Dateien wurden anschließend in R bereinigt. Präfixe wie `path:` und `Homo sapiens` wurden entfernt, um nur die relevanten Informationen abzubilden.

Da über die KEGG-API nur Entrez IDs und keine Gennamen bereitgestellt wurden, mussten die zugehörigen Gennamen nachträglich gemappt werden. Hierzu wurden über die Annotationsbibliothek `org.Hs.eg.db` gezielt die Entrez IDs mit den entsprechenden Gennamen verknüpft, die in den abgerufenen Pathways enthalten sind.

3.2 Aufbau der lokalen SQLite Datenbank

Auf Basis der KEGG-Daten sowie der Zuordnung von Entrez IDs zu Gennamen wurde eine lokale SQLite-Datenbank "`GeneDatabase.sqlite`" aufgebaut. Diese dient als semantisches Bindeglied zwischen den Genexpressionsdatensätzen und den Pathways. Die Entscheidung für eine relationale Datenbank fiel aufgrund des deutlich effizienteren Datenzugriffs im Vergleich zu CSV- oder Excel-Dateien. Die daraus resultierende Datenbank besteht aus drei Tabellen:

- Gene: Enthält die Entrez ID als Primärschlüssel, den vollständigen Gennamen sowie das Gensymbol

- Pathway: Enthält die KEGG- Pathway ID und den Namen des Pathways.
- Lookup_Gene_Pathway: Stellt eine N:M Relation zwischen Genen und Pathways dar.

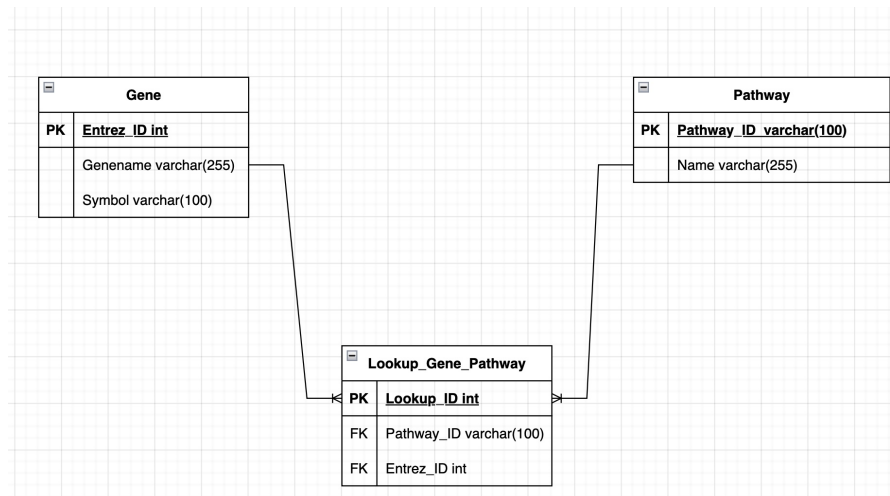


Abbildung 1: Eigene Darstellung: ERM Modell zum Aufbau der Datenbank

3.3 Funktionen zur Datenvorverarbeitung und Datenintegration

Die gesamte Programmlogik ist modular aufgebaut und gliedert sich funktional in vier zentrale Aufgabenbereiche: Datenvorverarbeitung zur Metadaten-Trennung, Coverage Analyse, Datenbankabfragen sowie die finale Datenintegration. Es existieren zwei spezialisierte Vorverarbeitungsfunktionen, die je nach Datentyp der ersten Spalte des Datensatzes aufgerufen werden: `preprocess_dataset_meta()` extrahiert numerische Entrez IDs, während `preprocess_dataset_meta_gennames()` genutzt wird, wenn Gennamen als `character` vorliegen. Beide Funktionen implementieren die Trennung von Messwerten und Metadaten über die Identifikation des Präfixes (`Meta_`). Dies ermöglicht es die Metainformationen separat von den eigentlichen Genexpressionsdaten zu speichern, sodass andere Teammitglieder für ihre Teilaufgaben direkt darauf zugreifen können.

3.3.1 Pathway Coverage

Bevor die eigentliche Datenintegration durchgeführt wird, kann mit der Funktion `analyze_pathways_coverage()` geprüft werden, wie gut die vom User ausgewählten Pathways durch den hochgeladenen Datensatz abgedeckt werden. Für jeden gewählten Pathway wird anhand einer Datenbankabfrage ermittelt, wie viele der in der Datenbank hinterlegten Gene im Datensatz vorhanden sind und wie viele fehlen. Die Funktion gibt eine Matrix mit den Spalten Gesamtanzahl, gefundene Gene, fehlende Gene sowie der prozentualen Abdeckung (Coverage) zurück. Diese Funktion dient als wichtige Entscheidungsgrundlage, um die Aussagekraft der nachfolgenden Analysen zu bewerten. Bei einer unzureichenden Abdeckung erhält der Nutzer somit die Möglichkeit, die Pathway Auswahl frühzeitig zu optimieren.

3.3.2 Datenintegration

Die Kernfunktion `run_data_integration()` verbindet alle Hilfsfunktionen und führt die semantische Filterung der Daten nach ausgewählten Pathways durch. Der Ablauf ist wie folgt:

- Der Datensatz wird anhand des Datentyps der ersten Spalte mit der passenden Funktion vorverarbeitet und von den Metadaten separiert.

- Über Datenbankabfragen werden alle relevanten Gene ermittelt, die den vom Nutzer gewählten Pathways zugeordnet sind. Es werden sowohl die IDs als auch die zugehörigen Gennamen in Vektoren gespeichert.
- Mittels der Funktion `extract_relevant_genes()` wird der Datensatz auf diese Gene reduziert. Werden keine Übereinstimmungen gefunden, bricht der Prozess mit einer Fehlermeldung ab.
- Die Funktion `rename_duplikate_genes()` behandelt mehrfach vorhandene IDs und macht sie über ein Suffix eindeutig (z. B. "51_1", "51_2"). Durch diese Eindeutigkeit können die IDs nun als Zeilenamen verwendet werden.
- Als Ergebnis wird eine benannte Liste zurückgegeben, die den gefilterten Datensatz, die Metadaten, den Genvektor sowie die zugehörigen Gennamen enthält.

3.4 Datensätze

Zum Testen der implementierten Funktionen wurden vier Genexpressions-Datensätze bereitgestellt. Zwei dieser Datensätze stammen aus dem TCGA-Register.

The Cancer Genome Atlas (TCGA) ist ein großes, öffentliches Forschungsprogramm, welches umfassende genomische Daten und klinische Metadaten von über 33 verschiedenen Krebsarten bereitstellt [5].

Zwei der für dieses Projekt verwendeten Testdatensätze stammen aus diesem Programm:

- **colon_vs_pancreas_meta.csv**: Datensatz, der Proben von Darm- und Bauchspeicheldrüsenkrebs vergleicht.
- **TCGA_kidney_unnormalized_meta.csv**: Datensatz, der die drei Nierenkrebs-Unterarten: papilläres, chromophobes und klarzelliges Nierenzellkarzinom enthält.

Ein weiterer zur Verfügung gestellter Datensatz ist der Golub-Datensatz: **GOLUB_microarray.csv**. Dieser enthält Genexpressionsdaten, die genutzt wurden, um Patienten mit zwei verschiedenen Arten von Blutkrebs zu klassifizieren: der akuten myeloischen Leukämie und der akuten lymphatischen Leukämie [15].

Der letzte zur Verfügung gestellte Datensatz ist der SHIPP-Datensatz: **SHIPP_microarray.csv**. Dieser enthält zwei Formen von Non-Hodgkin-Lymphomen: das diffuse großzellige B-Zell-Lymphom und das follikuläre Lymphom [35].

Die Datensätze wurden nach einem einheitlichen Schema aufbereitet und für alle Teammitglieder bereitgestellt: Während die erste Spalte immer die numerischen Entrez IDs oder Gennamen enthält, repräsentieren die darauffolgenden Spalten die einzelnen Patientenproben. Zur Erweiterung der Metadaten wurden für alle Datensätze zufällige Werte für Alter und Geschlecht generiert. Um die Metadaten sauber von den eigentlichen Genexpressionsdaten zu trennen, wurden diese einheitlich mit dem Präfix **Meta_** versehen.

4 Clusterverfahren

4.1 Arbeitsverteilung

Der Bereich der Clusteranalyse unterteilt sich in mehrere Unteraufgaben. Zunächst wurden die Daten auf die Weiterverarbeitung vorbereitet mit Hilfe einer Preprocessing-Funktion. Der Datensatz wurde daraufhin normalisiert, wobei unterschiedliche Normalisierungsfunktionen zur Auswahl standen. Aus dem normalisierten Datensatz wurde eine Distanzmatrix berechnet und anschließend ein Linkage-Verfahren angewendet. Dementsprechend mussten neben der Preprocessing-Funktion auch die Normalisierungs-, Distanz- und Linkage-Funktionen implementiert werden.

Die folgende Tabelle zeigt, welche Aufgaben von wem übernommen wurden und wie groß der Arbeitsaufwand war.

Aufgabe	Zuständig	Arbeitsaufwand
Datenvorbereitung	Rosina	6 Stunden
Normalisierungsfunktionen	Rosina	6 Stunden
Testen	Rosina	14 Stunden
Recherche	Rosina	5,5 Stunden
Bericht schreiben	Rosina	10 Stunden
Besprechungen	Rosina	19,5 Stunden
Complete-Linkage	Rosina	0,5 Stunden
Average-Linkage	Rosina	1 Stunde
Recherche zu Distanzfunktionen	Wiktorja	3 Stunden
Programmieren der Distanzfunktionen	Wiktorja	7 Stunden
Recherche zu Linkagefunktionen	Wiktorja	4 Stunden
Linkagefunktionen nach Lance-Williams	Wiktorja	12 Stunden
Testen (inkl. GUI- und Visualisierungsfunktionen)	Wiktorja	25 Stunden
Besprechungen	Wiktorja	19,5 Stunden
Protokollvor- und Nachbereitung	Wiktorja	3 Stunden
Berichterstellung (eigenes Kapitel)	Wiktorja	12 Stunden
Bericht zusammenfügen und Korrektur	Wiktorja	5 Stunden
Folien zusammenfügen und Korrektur	Wiktorja	1 Stunde
Repository-Pflege	Wiktorja	3 Stunden
Erstellen einer Library	Wiktorja	NOCH OFFEN

4.2 Allgemeines Vorgehen

Nach der Filterung der Genexpressionsdaten müssen sie für die anschließende Clusteranalyse vorbereitet werden. Rohdaten können fehlende Werte, unterschiedliche Wertebereiche oder andere Eigenschaften aufweisen, die die Qualität der Clusterbildung negativ beeinflussen. Eine sorgfältige Vorverarbeitung stellt daher sicher, dass die Daten in einer geeigneten Form für die Distanzberechnung und die Clusteranalyse vorliegen.

Die Vorverarbeitung besteht aus zwei Schritten. Zunächst werden die Daten auf Vollständigkeit und Gültigkeit überprüft sowie fehlende Werte behandelt. Anschließend erfolgt optional eine Normalisierung der Daten. Je nach gewählter Methode werden die Expressionswerte transformiert und skaliert um Unterschiede in den Wertebereichen zwischen den Genen zu reduzieren und die Vergleichbarkeit der Expressionsprofile zu verbessern. Dadurch wird eine zuverlässigere Grundlage für die nachfolgenden Clustering-Verfahren geschaffen.

Anfangs stellt jedes Element des Datensatzes (jede Beobachtung; dies bezieht sich auf die Gene, sowie auf die Patienten) ein eigenes Cluster dar. In jedem Schritt werden zwei Cluster zusammengefasst, bis schließlich alle Elemente in einem Cluster vereint sind. Das Zusammenfassen erfolgt mit einem bestimmten Linkage-Verfahren und basiert auf den Distanzen der Cluster zueinander. Zu diesem Zweck wird zunächst eine Distanzmatrix erstellt, die für alle Paare aus Elementen des Datensatzes die entsprechende Distanz enthält. Nach jedem Schritt werden die Distanzen für das neue Cluster aktualisiert. Je nach gewählten Linkage-Verfahren entstehen so unterschiedliche Aktualisierungen und damit auch unterschiedliche Cluster.

4.3 Datenvorbereitung

Vor der Normalisierung wird der Datensatz auf seine Eignung für die weitere Verarbeitung überprüft. Dies geschieht in der Funktion `prepare_data`, welches auffindbar im Github-Projekt unter `R/clustering/prepare_data.R` ist. Ziel dieser Funktion ist es, fehlerhafte oder unvollständige Eingabedaten frühzeitig zu erkennen und eine konsistente Datenstruktur für die anschließende Clusteranalyse sicherzustellen.

Als Erstes wird überprüft, ob der übergebene Datensatz leer ist oder keine Zeilen beziehungsweise Spalten enthält. Darüber hinaus wird kontrolliert, ob mindestens zwei Zeilen vorhanden sind, da eine Clusteranalyse mehrere Beobachtungen voraussetzt. Anschließend erfolgt eine Plausibilitätsprüfung der Werte. Falls der Datensatz nicht bereits als normalisiert gekennzeichnet wurde, wird überprüft, ob Werte kleiner als -1 enthalten sind. Solche Werte können darauf hinweisen, dass die Daten bereits transformiert oder skaliert wurden und eine erneute Normalisierung ungeeignet wäre. In diesem Fall wird die Verarbeitung mit einer entsprechenden Fehlermeldung abgebrochen. Für die weitere Verarbeitung wird der Datensatz anschließend in eine Matrix umgewandelt. Anschließend wird der ganze Inhalt zu numerischen Werten konvertiert. Dies stellt sicher, dass berechenbare Zahlen in der Matrix vorhanden sind. Nicht-numerische Werte, wie Buchstaben, werden dadurch zu fehlenden Einträgen (`NA`) umgewandelt. So können sie in der darauffolgenden `NA`-Behandlung mit entfernt werden. Der nächste Bestandteil der Datenvorbereitung ist die Behandlung fehlender Werte. Enthält eine Zeile fehlende Einträge, wird zunächst überprüft, ob mindestens ein gültiger numerischer Wert vorhanden ist. Ist dies der Fall, werden die fehlenden Werte, sofern welche in der Zeile vorhanden sind, durch den Mittelwert der numerischen Werte derselben Zeile ersetzt. Auf diese Weise bleiben möglichst viele Informationen erhalten und die `NA`-Werte werden ersetzt. Enthält eine Zeile ausschließlich fehlende Werte, wird die Verarbeitung abgebrochen, da keine Mittelwertberechnung möglich ist. Abschließend wird der Datensatz auf unendliche Werte (`Inf` beziehungsweise `-Inf`) überprüft. Da solche Werte bei den nachfolgenden Berechnungen zu Fehlern führen würden, wird die Verarbeitung in diesem Fall ebenfalls beendet. Nach Abschluss aller Prüfungen liegt ein vollständiger numerischer Datensatz vor, der für die anschließende Normalisierung und Clusteranalyse verwendet werden kann.

Während der Entwicklung dieser Funktion traten keine größeren Schwierigkeiten auf. Der Entwicklungsaufwand betrug etwa sechs Stunden, während die anschließende Testphase ungefähr vier Stunden in Anspruch nahm. Eine Besonderheit beim Testen der Funktion bestand darin, dass für bestimmte Fehlerfälle eigene, kleine Testdatensätze erstellt werden mussten. Dadurch konnten spezifische Fehler gezielt produziert und überprüft werden, da nicht alle möglichen Fehlerzustände regelmäßig in realen Datensätzen auftreten. Insbesondere die Behandlung von fehlenden Werten (`NA`-Werten) erforderte eine ausführliche Abstimmung. Hierzu wurde eine längere Zeitspanne einer Besprechung in Anspruch genommen, um festzulegen, unter welchen Bedingungen eine Fehlerbehandlung erfolgen soll und welche Vorgehensweise für die verschiedenen Fälle angemessen ist.

4.4 Normalisierung

Nach der Datenvorbereitung kann der Datensatz optional normalisiert werden.

Ziel der Normalisierung ist es, den Einfluss technischer Störfaktoren und systematischer Fehler auf die Messdaten zu reduzieren, sodass möglichst ausschließlich die biologische Variabilität der Genexpression erhalten bleibt. Solche unerwünschten Variationen können beispielsweise durch Unterschiede bei der Probenvorbereitung, leichte Schwankungen der Umgebungsbedingungen, den Abbau von Probenmaterial oder technische Einflüsse der verwendeten Messinstrumente entstehen. Generell sind diese Störquellen häufig weder direkt messbar noch exakt quantifizierbar,

deswegen stellt ihre Korrektur eine besondere Herausforderung dar [10].

Eine geeignete Normalisierung trägt daher wesentlich dazu bei, die Vergleichbarkeit der Daten zu verbessern und die Grundlage für eine zuverlässige Clusteranalyse zu schaffen. Für die in diesem Projekt entwickelte Funktion `normalization` stehen acht verschiedene Normalisierungsmethoden zur Verfügung, die nach dem beliebigen des Users ausgewählt werden können. Sie ist auffindbar im Projekt unter dem Pfad `R/clustering/normalization_methods.R`.

4.4.1 Keine Normalisierung

Bei dieser Methode werden die Daten unverändert weitergegeben. Sie eignet sich für Datensätze, die bereits vorverarbeitet oder mit einem externen Verfahren normalisiert wurden. Somit können auch normalisierte Datensätze visualisiert werden.

4.4.2 Nur Logarithmierung

In dieser Variante wird ausschließlich die Logarithmierung durchgeführt. Dies geschieht indem auf alle Expressionswerte nur eine Logarithmierung zur Basis 2 angewendet. Sie hilft den Dynamikbereich als auch die Schiefe der Verteilung zu reduzieren und führt so zu einer stärkeren Symmetrie, sowie nähert die Daten an eine Normalverteilung an. Dadurch erfüllen die Daten die Annahmen statistischer Analysen besser. Um Probleme mit Null- oder sehr kleinen Werten zu vermeiden, wird vor der Anwendung des Logarithmus üblicherweise eine kleine Konstante zu jedem Wert addiert [9]. Dies wurde in dieser Funktion auch durchgeführt mit einer Addierung von 1. Die Formel des Logarithmus ist wie folgt:

$$x'_1 = \log_2(x_1 + 1)$$

Durch das Logarithmieren werden große Expressionswerte abgeschwächt, während die absoluten Unterschiede zwischen den Genen weitgehend erhalten bleiben [11]. In dieser Methode erfolgt keine zusätzliche Skalierung der Daten. Die reine Log-Transformation ist sinnvoll, wenn der Nutzer die Wertekompression benötigt, aber die ursprüngliche Struktur der Daten möglichst unverändert beibehalten möchte. Alle darauffolgenden Logarithmen werden genau wie hier beschrieben durchgeführt.

4.4.3 Z-Score-Normalisierung

Variante mit Logarithmierung Diese Methode startet mit dem logarithmieren der Daten wie in 4.4.2 beschrieben. Anschließend erfolgt für jede Genzeile eine Z-Score-Normalisierung. Die Z-Score-Transformation ist eine weit verbreitete Normalisierungsmethode, bei der Merkmale (Gene) so skaliert werden, dass sie einen Mittelwert von 0 und eine Standardabweichung von 1 aufweisen. Für jedes Gen wird zunächst der Mittelwert (μ) und die Standardabweichung (σ) über alle Proben hinweg berechnet[9]. Jeder Wert (x) mithilfe der folgenden Formel standardisiert:

$$z_i = \frac{x'_1 - \mu}{\sigma}$$

Diese Transformation ermöglicht es, unterschiedliche Merkmale auf eine vergleichbare Skala zu bringen[9]. Der Nachteil ist, dass sie sensibel gegenüber Ausreißer ist, was zum Teil durch den Logarithmus unterbinden werden kann. Im eigentlichen R Code wird diese Formel mit der `scale`-Funktion durchgeführt. Da die Funktion spaltenweise arbeitet, wird der Datensatz zunächst transponiert, verarbeitet und abschließend erneut transponiert, um die ursprüngliche Ausrichtung wiederherzustellen.

Variante ohne Logarithmierung In dieser Methode wird ausschließlich die Z-Score Normalisierung durchgeführt. Im Gegensatz zur vorherigen beschriebenen Variante erfolgt keine vorherige Logarithmierung der Expressionswerte. Dadurch werden die ursprünglichen Expressionswerte direkt standardisiert.

Diese Variante eignet sich insbesondere für Datensätze, die bereits logarithmiert wurden oder bei denen bewusst auf eine zusätzliche Logarithmierung verzichtet werden soll.

4.4.4 Median-Zentrierung

Variante mit Logarithmierung Auch in dieser Methode wird zunächst die Logarithmierung durchgeführt (siehe 4.4.2). Anschließend wird jede Genzeile um ihren Median zentriert und mithilfe ihrer Spannweite skaliert. Das Ziel der Median-Zentrierung besteht darin, die Daten auf den Median der Verteilung jeder Probe zu zentrieren. Im Allgemeinen wird die Zentrierung anhand des Medians gegenüber dem Mittelwert bevorzugt, da der Median robuster gegenüber Ausreißern ist [10]. Der vollständige Normalisierungsprozess wird mit dieser Formel durchgeführt:

$$x''_i = \frac{x'_i - \text{Median}(x')}{\max(x') - \min(x') + \varepsilon}$$

Sie ist entstanden aus der Kombination der Median-Zentrierung und der Range-Skalierung. Die Formel der Median-Zentrierung ist nach [10]:

$$x''_i = x'_i - \text{Median}(x')$$

Bei der Range-Skalierung werden die zentrierten Werte durch die Spannweite, also die Differenz zwischen dem Maximum und Minimum einer Genzeile, dividiert. Dadurch werden Unterschiede in den Wertebereichen der Gene reduziert, sodass alle Gene unabhängig von ihrer ursprünglichen Variabilität einen vergleichbaren Einfluss auf die nachfolgende Analyse besitzen. Da die Spannweite den biologischen Variationsbereich der Daten repräsentiert, orientiert sich die Skalierung an diesem biologischen Wertebereich [41]. Dargestellt wird es durch diese Formel nach [41]:

$$x^*_{ij} = \frac{x_{ij} - \bar{x}_i}{x_{i,\max} - x_{i,\min}}$$

Das ε in der vollständigen Formel wird definiert mit $\varepsilon = 1e^{-8}$ und ist eine kleine Konstante, die die Teilung durch Null verhindert. Ein Nachteil der Range-Skalierung besteht jedoch darin, dass die Spannweite ausschließlich auf den beiden Extremwerten basiert und daher empfindlich gegenüber Ausreißern ist [41]. Allerdings wird dieser Schwäche zum Teil durch die Median-Zentrierung entgegengewirkt.

Variante ohne Logarithmierung In dieser Methode wird nur die oben beschriebene Median-Zentrierung durchgeführt, während das Logarithmieren ausgelassen wird. Wie schon bereits erklärt ist dies von besonderem Nutzen, wenn die Originalstruktur des Datensatzes von großer Bedeutung, oder der Datensatz schon vorbereitet worden ist.

4.4.5 MAD-Normalisierung

Variante mit Logarithmierung Diese Methode verwendet ebenfalls zunächst die Logarithmierung (siehe 4.4.2). Anschließend erfolgt eine robuste Skalierung mithilfe der Median Absolute Deviation (MAD):

$$x_i'' = \frac{x_i' - \text{Median}(x')}{\text{MAD}(x') + \varepsilon}$$

Dabei bezeichnet

$$\text{MAD}(x) = \text{Median}(|x - \text{Median}(x)|)$$

die Median Absolute Deviation. $\varepsilon = 1e - 8$ ist erneut eine kleine Konstante, die eine Division durch Null verhindert. Anstelle der Standardabweichung wird die Median Absolute Deviation (MAD) als robustes Maß für die Variabilität verwendet. Sie entspricht dem Median der absoluten Abweichungen aller Beobachtungen vom Stichprobenmedian [34].

Im Vergleich zur Standardabweichung ist MAD deutlich robuster gegenüber Ausreißern und eignet sich daher insbesondere für Datensätze mit einzelnen extremen Expressionswerten [21].

Variante ohne Logarithmierung Diese Methode entspricht der im vorherigen Paragraph beschriebenen MAD-Normalisierung, jedoch ohne vorherige Logarithmierung. Die ursprünglichen Expressionswerte werden direkt anhand der Median Absolute Deviation (MAD) skaliert. Diese hat vor allem Nutzen, wenn Reduktion der Verteilung im Datensatz nicht nötig ist und nur der Normalisierungseffekt benötigt wird.

4.4.6 Herausforderungen bei der Implementierung

Der gesamte Programmieraufwand für die Funktion `normalization` betrug etwa sechs Stunden. In dieser Zeit wurden sowohl die bestehende Struktur überarbeitet als auch neue Normalisierungsmethoden implementiert. Die eigentliche Implementierung der Funktion erfolgte vergleichsweise zügig. Deutlich zeitintensiver war hingegen das Testen, welches insgesamt etwa zehn Stunden in Anspruch nahm.

Während der Testphase trat bei einem von einem weiteren Tester verwendeten Datensatz ein Fehler auf, dessen Ursache zunächst unklar war. Nach eingehender Analyse stellte sich heraus, dass der Datensatz bereits vorab normalisiert worden war. Zu diesem Zeitpunkt bot das Programm jedoch keine Möglichkeit, bereits vorbereitete Datensätze korrekt zu verarbeiten oder dem Benutzer explizit die Auswahl eines vorab normalisierten Datensatzes zu ermöglichen. Aus diesem Grund wurden entsprechende Optionen für den Benutzer ergänzt sowie zusätzliche Fehlerbehandlungen implementiert.

In der Endphase des Projekts wurde außerdem festgestellt, dass das Clustering beziehungsweise die Darstellung der Cluster nicht korrekt funktionierte. Daraufhin wurde die gesamte Normalisierungsfunktion erneut überprüft und ausführlich getestet. Gleichzeitig wurde der Datensatz, der den Fehler besonders deutlich sichtbar machte, nochmals analysiert. Dabei konnte die eigentliche Fehlerursache identifiziert und behoben werden. Im Zuge dieser Überarbeitung fiel zudem die Entscheidung, den Funktionsumfang zu erweitern und zusätzlich Normalisierungsmethoden ohne vorgeschaltete Logarithmierung zu implementieren.

Einen weiteren unerwartet hohen Zeitaufwand verursachte die Recherche geeigneter Normalisierungsmethoden (5,5 Stunden). Zwar wird in wissenschaftlichen Veröffentlichungen häufig angegeben, dass Verfahren wie beispielsweise die Z-Score-Normalisierung verwendet wurden, jedoch fehlen oftmals detaillierte Beschreibungen der konkreten Implementierung oder eine Begründung für die Wahl der jeweiligen Methode.

4.5 Berechnung der Distanzmatrix

Für das Zusammenfügen der Cluster muss zunächst bestimmt werden, wie ähnlich, bzw. nah diese zueinander sind. Da es sich bei den in diesem Projekt verwendeten Genexpressionsdaten um kontinuierliche Daten handelt, wird ihre Nähe mit Hilfe von Distanzmaßen bestimmt. Für ein Distanzmaß δ_{ij} muss die Dreiecksungleichung gelten, d.h. für drei Elemente i , j und m :

$$\delta_{ij} + \delta_{im} \geq \delta_{jm}$$

Durch Berechnen der zeilenweisen Distanzen δ_{ij} , für welche diese Bedingung gilt, entsteht eine symmetrische Distanzmatrix $\mathbf{D}$, wobei $\delta_{ii} = 0$ für alle i gilt [13]. Es muss sowohl eine Distanzmatrix für die Gene als auch für die Patienten berechnet werden. Im zweiten Fall wird der Datensatz zunächst transponiert, so dass zeilenweise gearbeitet werden kann.

Es gibt eine Vielzahl an Distanzmaßen, die verwendet werden können. In dem Projekt wurden die folgenden sechs Distanzfunktionen implementiert: euklidische Distanz, Manhattan-Distanz, Minkowski-Distanz, Canberra-Distanz, Pearson-Korrelation und Winkeldistanz (Angular Separation). Eine Übersicht der Distanzfunktionen befindet sich in Tabelle 1. Hierbei sind x_{ik} und x_{jk} jeweils die k -te Variablen der p -dimensionalen Beobachtungen der Variablen i und j . Die Auswahl entstammt Everitt et al. [13], verzichtet allerdings auf Gewichte.

Die euklidische Distanz (D1) gilt häufig als das gängigste Distanzmaß. Sie hat die Eigenschaft, dass die berechnete Distanz als ein Abstand zwischen zwei Punkten im p -dimensionalen Raum verstanden werden kann [13]. Die Manhattan-Distanz (D2) ist ähnlich, definiert den Abstand allerdings ausschließlich über geradlinige Verbindungen. D1 und D2 sind Spezialfälle der Minkowski-Distanz (D3), für $r = 2$, bzw. $r = 1$. Dem Nutzer wird bei Auswahl von D3 die Möglichkeit gegeben, einen eigenen Wert für r einzugeben, wobei Werte < 1 nicht zulässig sind.

Die Canberra-Distanz (D4) ist besonders sensitiv für kleine Veränderungen nahe 0 [13].

Mit D5 und D6 stehen auch zwei Korrelationsmaße als Distanzfunktionen zur Verfügung. Für die Korrelation ϕ_{ij} gilt $-1 \leq \phi_{ij} \leq 1$, wobei 1 die größte positive Korrelation und -1 die größte negative Korrelation darstellt. Die Korrelation kann in eine Distanz δ_{ij} im Intervall $[0, 1]$ umgerechnet werden. Korrelationsmaße eignen sich für Daten, die auf der gleichen Skala gemessen wurden, sowie deren exakten Werte nur insofern von Interesse sind, als sie Aufschluss über das relative Profil geben. So hat z.B. die Winkeldistanz (D6) die Eigenschaft, dass nur dann eine minimale Distanz von 0 zwischen zwei Zeilen entsteht, wenn die Werte der einen Zeilen Mehrfache der Werte der anderen Zeile sind.

Im Projekt wurde eine gemeinsame Funktion für die Berechnung der Distanzmatrix in C++ implementiert. Dafür wurde ein separates Repository erschaffen und als Paket exportiert. Das Paket `Rcpp` [12] ermöglicht die Einbindung des C++-Codes in eine R-Umgebung. So kann die C++ Funktion in R ausgeführt werden. Hierzu muss das entsprechende Paket `distRcpp` (Version 0.0.9001) aufgerufen werden. Die eigentliche Funktion heißt `dist_cpp` und bekommt drei Parameter übergeben: eine numerische Matrix `mat` (den normalisierten Datensatz), einen String `method` für das gewählte Distanzmaß, und einen Integer `p`. Letzterer entspricht dem Parameter r in der Minkowski-Distanz (D3). Die Implementierung in C++ folgt weithin den mathematischen Formeln und ist daher vergleichbar unkompliziert. Vor allem die Einbindung des Codes als separates Paket war jedoch zeitaufwendig und führte zunächst zu einigen Fehlern, die primär die Installation des Pakets betrafen. Als diese schließlich gelöst wurden, konnte `dist_cpp` problemlos aufgerufen werden.

Nr.	Name	Funktion
D1	Euklidische Distanz	$d_{ij} = (\sum_{k=1}^p (x_{ik} - x_{jk})^2)^{1/2}$
D2	Manhattan-Distanz	$d_{ij} = \sum_{k=1}^p x_{ik} - x_{jk} $
D3	Minkowski-Distanz	$d_{ij} = (\sum_{k=1}^p x_{ik} - x_{jk} ^r)^{1/r}$ mit $r \geq 1$
D4	Canberra-Distanz	$d_{ij} = \begin{cases} 0 & \text{wenn } x_{ik} = x_{jk} = 0 \\ \sum_{k=1}^p x_{ik} - x_{jk} / (x_{ik} + x_{jk}) & \text{sonst} \end{cases}$
D5	Pearson-Korrelation	$\delta_{ij} = \frac{1 - \phi_{ij}}{2},$ $\phi_{ij} = \frac{\sum_{k=1}^p (x_{ik} - \bar{x}_{i\cdot})(x_{jk} - \bar{x}_{j\cdot})}{[\sum_{k=1}^p (x_{ik} - \bar{x}_{i\cdot})^2 \sum_{k=1}^p (x_{jk} - \bar{x}_{j\cdot})^2]^{1/2}},$ $\bar{x}_{i\cdot} = \frac{1}{p} \sum_{k=1}^p x_{ik}$
D6	Winkeldistanz	$\delta_{ij} = \frac{1 - \phi_{ij}}{2},$ $\phi_{ij} = \frac{\sum_{k=1}^p x_{ik} x_{jk}}{(\sum_{k=1}^p x_{ik}^2 \sum_{k=1}^p x_{jk}^2)^{1/2}}$

Tabelle 1: Implementierte Distanzmaße

4.6 Linkagefunktionen

Die eigentliche Clusterung erfolgt mit der Linkage-Funktion `hierarchichal_clustering`. Diese Funktion erhält als Parameter eine Distanzmatrix `d_mat` (siehe Kapitel 4.5), den Namen des gewünschten Linkage-Verfahrens `method` als String, sowie optional eine Liste mit vier Parametern `custom_params`. Standardmäßig wird als Methode `"single"` und für die Parameter `NULL` übergeben.

Das Ergebnis dieser Funktion ist eine Liste mit zwei Elementen, die für die Visualisierung der Ergebnisse benötigt werden. Das erste Element ist `matched_at`, ein Vektor, der die Werte enthält, an denen die Cluster zusammengefügt wurden. Dies entspricht der minimalen Distanz an jedem Schritt. Das zweite Element der Output-Liste ist `merge`, eine $n - 1$ mal 2 Matrix, die in jeder Zeile die zusammengeführten Cluster-Indizes enthält.

Als Methoden stehen Single-Linkage (`single`), Average-Linkage (`average`) und Complete-Linkage (`complete`), sowie eine benutzerdefinierte Linkage-Funktion (`custom`) zur Verfügung. Nach jeder Zusammenführung zweier Cluster wird die Distanzmatrix entsprechend der gewählten Linkage-Methode aktualisiert.

4.6.1 Linkagefunktion nach Lance-Williams

Die Methode `custom` implementiert die allgemeine Formel von Lance und Williams [25]. Werden zwei Cluster i und j zu einem neuen Cluster k zusammengefasst, so kann die Distanz des neuen

Clusters k zu einem weiteren Cluster h durch diese Funktion aktualisiert werden:

$$d_{hk} = \alpha_i d_{hi} + \alpha_j d_{hj} + \beta d_{ij} + \gamma |d_{hi} - d_{hj}|$$

Die vier Parameter α_i , α_j , β und γ bestimmen die Aktualisierung der Distanz. Erst ihre jeweilige Kombination definiert ein konkretes Linkage-Verfahren, wie Single-, Complete- oder Average-Linkage. Tabelle 2 enthält eine Übersicht der erforderlichen Parameter für diese drei Standardverfahren.

Die Implementierung basiert vollständig auf dieser allgemeinen Formel. Dadurch kann die Aktualisierung der Distanzmatrix unabhängig vom konkreten Linkage-Verfahren mit derselben Implementierung erfolgen; lediglich die Parameter unterscheiden sich zwischen den Verfahren. Für die Methoden **single**, **average** und **complete** werden die zugehörigen Parameter automatisch gesetzt. Wird hingegen **custom** gewählt, müssen die vier Parameter vom Benutzer über die Eingabemaske angegeben werden. Die Eingabe wird lediglich auf numerische Werte überprüft. Dynamische Ausdrücke wie n_i/n_k , die beispielsweise für Average-Linkage verwendet werden, können daher nicht angegeben werden. Ebenso erfolgt keine Überprüfung hinsichtlich der theoretischen Sinnhaftigkeit der gewählten Parameter.

4.6.2 Bedeutung der Parameter

Die Parameter α_i und α_j gewichten den Beitrag der ursprünglichen Cluster i und j auf die neue Distanz d_{hk} . Gilt $\alpha_i = \alpha_j$, so werden beide Cluster gleich gewichtet, wie es bei Single- und Complete-Linkage der Fall ist. Beim Average-Linkage-Verfahren werden die Gewichte proportional zu der jeweiligen Clustergröße gewählt. Hierbei entsprechen n_i und n_j jeweils der Anzahl der Elemente im Cluster i , bzw. j . Es gilt $n_k = n_i + n_j$.

Der Parameter β gewichtet zusätzlich die Distanz d_{ij} zwischen den beiden zusammengeführten Clustern. Dadurch kann die Lage der neu berechneten Distanz beeinflusst werden. Für die drei implementierten Standardverfahren gilt $\beta = 0$, daher wirkt sich der Term nicht auf das Ergebnis aus. Lance und Williams diskutieren jedoch die zentrale Rolle dieses Parameters in der sogenannten *Flexiblen Strategie*, wobei β im Bereich $-1 \leq \beta < 1$ betrachtet wird, so dass weitere Linkage-Verfahren entstehen.

Der Parameter γ gewichtet die absolute Differenz zwischen d_{hi} und d_{hj} . Für die Werte $\gamma = -0,5$, bzw. $\gamma = 0,5$ reduziert sich die Lance-Williams-Formel auf das Minimum, bzw. das Maximum der beiden Distanzen. Dadurch erhält man unmittelbar Single-Linkage, bzw. Complete-Linkage. Für $\gamma = 0$ hingegen entfällt dieser Term vollständig, wie es z.B. beim Average-Linkage der Fall ist.

Linkage-Verfahren	α_i	α_j	β	γ	Eigenschaft
Single-Linkage	0,5	0,5	0	-0,5	Minimum
Complete-Linkage	0,5	0,5	0	0,5	Maximum
Average-Linkage	n_i/n_k	n_j/n_k	0	0	gewichteter Mittelwert

Tabelle 2: Parameter und Eigenschaften der implementierten Linkageverfahren

4.6.3 Eigenschaften der Standard-Linkageverfahren

Das Single-Linkage-Verfahren fügt Cluster mit einer minimalen Distanz zusammen und wird daher auch als Nearest-Neighbor-Verfahren bezeichnet. Es handelt sich um ein raumverkleinerndes Verfahren, das zum sogenannten *Chaining*-Effekt neigt. Hierbei werden einzelne Objekte oder kleinere Cluster schrittweise zu langen Clusterketten verbunden.

Das Complete-Linkage-Verfahren stellt den Gegenpol zum Single-Linkage dar, da es auf einer maximalen Distanz basiert. Es wird auch als Furthest-Neighbor-Verfahren bezeichnet. Dementsprechend verfolgt es eine raumvergrößernde Strategie, die häufig zu kompakteren und stärker voneinander getrennten Clustern führt.

Das Average-Linkage-Verfahren ist ein Beispiel für ein konservierendes Verfahren, das den Raum weder verkleinert noch vergrößert. Es basiert auf dem gewichteten Mittelwert, was alleine durch die Parameter α_i und α_j ausgedrückt wird, während die beiden anderen Terme wegfallen. Damit stellt es einen Kompromiss zwischen Single- und Complete-Linkage dar.

4.6.4 Fazit

Diese drei implementierten Verfahren können somit als Spezialfälle der allgemeinen Lance-Williams-Funktion verstanden werden. Die benutzerdefinierte Linkagefunktion erlaubt darüber hinaus die freie Wahl der vier Parameter und damit die Implementierung weiterer hierarchischer Linkage-Verfahren.

5 Visualisierung

Für die Visualisierung wurden eine Heatmap und zwei Dendrogramme erstellt. Dabei zeigt ein Dendrogramm die Patienten und das andere die Gene. Der Arzt kann sich am Ende die einzelnen Dendrogramme sowie ein Grafikpanel, welches aus der Zusammensetzung der einzelnen Komponenten besteht, anschauen.

5.1 Das Dendrogramm

Unter einem Dendrogramm versteht man ein **Baumdiagramm**, welches die hierarchischen Beziehungen zwischen Objekten darstellt und diese in zusammengehörige, ähnliche Gruppen kategorisiert. Es macht somit Zusammenhänge sichtbar, die sich aus einer reinen Distanz- oder Ähnlichkeitsmatrix kaum ablesen lassen: Auf einen Blick wird erkennbar, welche Objekte früh zu Clustern verschmelzen, welche isoliert bleiben und wie sich diese Gruppen auf höheren Ebenen weiter zusammenfassen lassen. Auf diese Weise schlägt das Dendrogramm eine Brücke zwischen komplexen mathematischen Daten und dem, was ein Mensch auf einen Blick erfassen kann. Sein Aufbau folgt dabei einer klaren, streng hierarchischen Baumstruktur.

Am unteren Ende der Baumstruktur liegen die **Blätter**, die jeweils ein einzelnes Untersuchungsobjekt repräsentieren und den Ausgangspunkt der Clusterung bilden.

Von diesen ausgehend symbolisieren die **internen Knoten** des Baumes die Verschmelzungspunkte, an denen kleinere Cluster oder einzelne Datenpunkte basierend auf ihrer Ähnlichkeit zusammengeführt werden.

Der **Wurzelknoten** des Graphen steht schließlich als Repräsentation der Gesamtheit aller analysierten Daten ganz oben und bildet als ein allumfassendes Cluster somit die höchste Hierarchieebene. Die dabei entstehenden **Äste** dienen als Verbindungslinien zwischen den verschiedenen Knoten. Die entscheidende Rolle spielt hierbei die Höhe eines Knotens beziehungsweise die Länge des jeweiligen Astes, denn sie ist proportional zur Unähnlichkeit der verbundenen Elemente: Je höher ein Knoten im Diagramm liegt, desto größer ist die Distanz und desto unähnlicher sind die darunter liegenden Cluster [19, 26].

5.1.1 Das Dendrogramm im Projektkontext

Die oben beschriebene Struktur wird im vorliegenden Dashboard auf einen Datensatz zur Genexpression angewendet, dessen Matrix in den Zeilen die Gene und in den Spalten die Patienten führt. Da sie sich in beide Richtungen lesen lässt, wird die Clusterung zweimal durchgeführt: Das Dendrogramm der Gene zeigt, welche Gene über alle Patienten hinweg ein ähnliches Expressionsmuster aufweisen, während das Dendrogramm der Patienten jene mit einem ähnlichen Gesamtprofil zusammenfasst. Beide sind im kombinierten Grafikpanel um eine gemeinsame Heatmap herum angeordnet, das Dendrogramm der Patienten darüber und das der Gene links daneben. Die Reihenfolge der Blätter gibt dabei jeweils vor, wie die Zeilen und Spalten der Heatmap angeordnet sind.

5.1.2 Die Eingabe: Das Cluster-Objekt

Um das Dendrogramm überhaupt realisieren zu können, benötigt die Visualisierung eine geeignete Datengrundlage. Diese liefert das Cluster-Team in Form eines Cluster-Objekts, das an die Struktur von `hclust` angelehnt ist [32]. Den Ablauf der Clusterung protokolliert die Merge-Matrix. Bei n Objekten hält sie in $n - 1$ Zeilen fest, welche beiden Einheiten in jedem Schritt verschmolzen wurden. Das Vorzeichen kodiert dabei die Art des Elements. Ein negativer Wert steht für ein ursprüngliches Blatt, dessen Betrag den Index angibt, ein positiver Wert für einen bereits gebildeten Teilbaum, wobei die Zahl auf die Zeilennummer des früheren Merge-Schritts

verweist. Der Baum lässt sich so vollständig aus der Tabelle ableiten, mit der Wurzel in der letzten Zeile.

Ergänzt wird das durch den Vektor `matched_at`, der zu jedem Schritt die Distanz zwischen den beiden zusammengeführten Einheiten festhält und dem Dendrogramm damit seine Höhe verleiht. Der interne Knoten des n -ten Merge-Schritts wird also auf der Höhe `matched_at[n]` gezeichnet.

```
> print(cluster_pat$merge[1:4, ])
[,1] [,2]
[1,] -60 -63
[2,] -21 -68
[3,] -61 -66
[4,]  2 -69

> print(cluster_pat$matched_at[1:4])
[1] 0.8383706 0.9801695 1.0009455 1.0106337
```

Die ersten drei Zeilen verschmelzen jeweils zwei ursprüngliche Objekte, in Schritt 1 etwa die Patienten 60 und 63 auf Höhe 0.838. Zeile 4 zeigt den gemischten Fall, denn die positive 2 verweist auf den Teilbaum aus Schritt 2, der hier mit Patient 69 zusammengeführt wird. Die wachsenden Höhen folgen daraus, dass jeder Schritt das jeweils nächstgelegene Paar zusammenführt.

5.1.3 Architektur der Implementierung

Der Aufbau der Implementierung trennt Berechnung und Darstellung konsequent voneinander. Eine Datenschicht ermittelt, was gezeichnet werden muss, eine Farbschicht bestimmt die Einfärbung, und die Umsetzung in die jeweilige Grafikbibliothek übernehmen austauschbare Renderer.

Die Datenschicht Den Anfang macht die Funktion `build_tree()`, die aus der Merge-Matrix einen verschachtelten Binärbaum aufbaut. Dabei tragen nur Blätter eine `id`, während interne Knoten `id = NULL` führen, und dieses eine Merkmal unterscheidet beide in allen späteren Traversierungen. Auf diesem Baum arbeitet `get_order_vector()`, das ihn in Inorder durchläuft und so die Blattreihenfolge von links nach rechts als Vektor aus Integer-IDs ausliest. Dieselbe Reihenfolge legt zugleich die Achsenanordnung der Heatmap fest, sodass beide Darstellungen auf derselben Grundlage beruhen und jeder Ast über der zugehörigen Zeile beziehungsweise Spalte steht.

Die Funktion `calculate_coords()` bestimmt anschließend rekursiv die Position jedes Knotens. Blätter erhalten die y -Koordinate 0, ihre x -Koordinate stammt aus dem Order-Vektor. Interne Knoten sitzen auf der Höhe, auf der ihre Teilbäume zusammengeführt werden. Ihre x -Koordinate ergibt sich als Mittelwert der beiden Kindpositionen.

Aus diesen Positionen wiederum sammelt `draw_segments()` schließlich die Liniensegmente sowie die Blatt-Metadaten aus ID, Position und Klasse ein. Pro internem Knoten entstehen dabei vier Segmente, denn je Kind entsteht eine Hälfte der Verbindungslinie samt Vertikale in dessen Klassenfarbe, wodurch ein Ast so weit farbig bleibt, wie die Klassen der Teilbäume übereinstimmen.

Die Ergebnisse werden in `generate_dendro_data()` als Objekt aus `draw_result` und `max_height` gesammelt, ohne jeden Bezug zu einer Grafikbibliothek. Da die Daten unabhängig vom Renderer vorliegen, sind die nachgelagerten Schichten austauschbar.

Die Farbschicht und die Renderer Für die Einfärbung der entstandenen Segmente sorgt `get_color()`, die jedes Klassenlabel auf einen Hex-Wert abbildet. Die Klasse `Default`, die einem Knoten zugeordnet wird, wenn seine Teilbäume nicht derselben Klasse angehören, erhält dabei immer Schwarz. Reicht die Palette nicht aus oder ist sie unbekannt, greift eine Ersatzpalette. Gibt es hingegen gar keine echte Klasse einzufärben, schreibt `get_color()` nur die Zuordnung von `Default` auf Schwarz in den Vektor. Alles andere fällt beim Zeichnen auf diese eine Farbe zurück, und die Darstellung bleibt durchgehend schwarz.

Für die finale Visualisierung sind zwei verschiedene Plot-Funktionen implementiert:

`plot_dendro_ggplot()` erzeugt die statische Variante für den PDF-Bericht, indem sie ein `ggplot`-Objekt Schicht für Schicht aufbaut [45]

`plot_dendro_plotly()` hingegen erzeugt die interaktive Variante für das Dashboard. Dazu baut die Funktion die Segmente als native Linien-Traces auf und setzt die um 90 Grad gedrehten Beschriftungen als Annotationen [36]

Beide Funktionen nehmen dasselbe Datenobjekt entgegen und setzen auf dieselbe Farblogik, wodurch Bericht und Oberfläche auf identischer Blattreihenfolge und Struktur arbeiten. Über den Parameter `names_vector` werden die Integer-IDs an dieser Stelle in der jeweiligen Plot-Funktion zu Klarnamen aufgelöst.

Das Grafikpanel lag ursprünglich im Aufgabenbereich meiner Teampartnerin. Nachdem mehrere Versuche gescheitert waren, Dendrogramm und Heatmap zusammenzuführen, übernahm ich diese Aufgabe aus Zeitgründen. `grafikpanel()` bezieht die Heatmap aus der Wrapper-Funktion meiner Teampartnerin und zeichnet die Dendrogramme auf Grundlage von `generate_dendro_data()` und `get_color()` unmittelbar neu. Die Anordnung entsteht über manuell gesetzte Achsen-Domains innerhalb eines einzigen `plot_ly()`-Objekts, wobei gekoppelte Achsen dafür sorgen, dass Zoom und Verschiebung synchron auf Heatmap und zugehörigem Dendrogramm wirken.

5.1.4 Herausforderungen und Entscheidungen

Flexibilität statt Spezialfälle Da das Dendrogramm zweimal mit unterschiedlicher Semantik aufgerufen wird, einmal für Gene und einmal für Patienten, wäre es naheliegend gewesen, diese Unterscheidung über einen internen `type`-Parameter abzubilden. Stattdessen fiel die Entscheidung bewusst darauf, Namensvektor und Klassenlabels von außen zu übergeben. Das hält die Zeichenfunktion frei von Sonderfällen und öffnet sie für beliebige weitere Anwendungen, verlagert aber zugleich die Verantwortung für korrekte Eingaben auf den Aufrufer.

Entscheidungen in der Datenschicht Mehrere Details der Implementierung sind bewusst auf Effizienz hin ausgelegt, was sich an drei Entscheidungen zeigt.

Für die Blattreihenfolge fiel die Wahl auf eine In-Order-Traversierung, die zusammengehörige Blätter benachbart hält, ohne dass sich Äste kreuzen. Bei den Namen wäre es möglich gewesen, sie direkt im Baum mitzuführen. Stattdessen speichern Baum und Order-Vektor ausschließlich Integer-IDs, denn Integer-Vergleiche sind über tausende Blätter hinweg deutlich billiger als String-Vergleiche, und die Auflösung zu Klarnamen erfolgt erst in der Zeichenschicht.

Auch die Berechnung von Koordinaten und Segmenten ist auf möglichst wenige Durchläufe ausgelegt. `calculate_coords()` liefert in einem einzigen Durchlauf den vollständigen Koordinatenbaum inklusive Kindpositionen, sodass `draw_segments()` diese direkt auslesen kann. `draw_segments()` wiederum sammelt die Segmente zunächst in Listen und führt sie erst am Ende in einem einzigen `rbind()` zusammen, da ein Data Frame in R sonst bei jedem Anfügen neu kopiert werden würde und der Aufwand quadratisch mit der Knotenzahl ansteigen würde.

Darstellung von großen Datensätzen Mit wachsender Blattzahl überlappen die Achsenbeschriftungen, und bei fester Strichstärke laufen die Äste ineinander, was sich vor allem beim PDF-Export bemerkbar macht. Die ggplot-Variante fängt das über drei dynamisch berechnete Größen ab. Schriftgröße und Linienstärke richten sich nach der Elementanzahl, die untere Marge nach der Länge des längsten Namens. Da es sich um eine Vektorgrafik handelt, bleibt der Export auch bei kleiner Schrift beliebig zoombar. Die Plotly-Variante übernimmt dieselbe Anpassung und ergänzt sie um Zoom sowie einen punktgenauen Hover mit Name und Klasse.

Der ursprüngliche Ansatz und die kurzfristige Umstellung auf plotly Die Umsetzung war zu Projektstart bewusst auf ggplot ausgerichtet. Die Dendrogramme sollten als SVG exportiert und vom GUI-Team direkt eingebunden werden, während das kombinierte Panel über `patchwork` entstehen sollte, und das alles ohne Codeänderung an den Kernfunktionen.

Beide Pläne wurden hinfällig, als das GUI-Team gegen Projektende die Einbindung in die Oberfläche per Plotly realisieren wollte. Durch die Anwendung von `ggplotly()` auf die fertigen ggplot-Objekte hatte, durch eine fehlerhafte Konvertierung, zur Folge, dass die Grafiken nicht mehr korrekt dargestellt wurden.

Für das kombinierte Panel scheiterte zunächst auch der Versuch, die mit Konvertierungsfunktionen aufbereiteten Objekte per `subplot()` zusammenzuführen. Daraufhin wurde das Dendrogramm vollständig als komplett neuer Plotly-Renderer umgebaut, doch auch der erneute Versuch, das Panel über `subplot()` zusammenzusetzen, führte nicht zum Ziel. Die finale Lösung ist daher ein hybrider Ansatz: Die Heatmap stammt weiterhin aus der Wrapper-Funktion, während die Dendrogramme im Panel nativ neu aufgebaut werden. Auch hier erwies sich die klare Schichtentrennung erneut als hilfreich.

5.1.5 Fazit

Abschließend stellte diese Umstellung wohl die größte Herausforderung des Projekts dar, und die kurzfristigen Anpassungen sind der Architektur bis heute anzumerken. So werden Namen etwa an drei Stellen aufgelöst, weil Renderer und Panel nach und nach ergänzt wurden. Für die Berechnung der Dendrogramme selbst hätte sich zudem eine Memorisierung angeboten, um wiederholte Traversierungen bei großen Bäumen durch das Zwischenspeichern bereits berechneter Koordinaten zu vermeiden. Auch das Grafikpanel ist eher eine pragmatische Lösung als das strukturelle Optimum, denn es dupliziert unter anderem die Logik für Färbung und Hover, die im eigenständigen Plotly-Renderer bereits existiert, und zentrale Layout-Werte wie die Achsendomänen sind fest codiert statt dynamisch aus der Datengröße abgeleitet. Mit mehr Zeit wäre ein gemeinsamer, parametrisierter Unterbau für Renderer und Panel sinnvoll gewesen.

5.1.6 Arbeitsaufwand

Aufgabe	Zuständig	Arbeitsaufwand
Recherche	Dominik	3 Stunden
Besprechungen	Dominik	19.5 Stunden
Bericht schreiben	Dominik	4 Stunden
Team Besprechungen	Dominik	8 Stunden
Datenstruktur aufsetzen	Dominik	5 Stunden
Implementierung der Plot - Funktion für das Dendrogramm	Dominik	3 Stunden
Einpflegen der Farblogik	Dominik	1 Stunden
PDF Speicherung	Dominik	1 Stunden
Neuausrichtung auf Plotly	Dominik	15 Stunden
Testen	Dominik	7 Stunden

5.2 Heatmap

5.2.1 Theoretische Grundlagen

Eine Heatmap dient der grafischen Darstellung numerischer Daten in Form einer Matrix, wobei jeder Messwert durch eine Farbe repräsentiert wird. In der Genexpressionsanalyse werden dabei typischerweise die Zeilen durch Gene und die Spalten durch Patienten beziehungsweise Proben dargestellt. Die Expressionshöhe wird über eine Farbskala codiert, wodurch Unterschiede und ähnliche Expressionsmuster visuell erkennbar werden [16].

Durch die Kombination mit einer hierarchischen Clusteranalyse können Gene und Patienten entsprechend ihrer Ähnlichkeit angeordnet werden. Dadurch entstehen zusammenhängende Bereiche mit ähnlichen Expressionsprofilen, wodurch biologische Muster oder mögliche Untergruppen leichter identifiziert werden können [16].

Bei der Implementierung müssen die Expressionswerte als numerische Matrix vorliegen und die Reihenfolge der Gene sowie Patienten aus der Clusteranalyse übernommen werden. Zusätzlich ist eine geeignete Farbpalette erforderlich, welche Unterschiede unverzerrt darstellt und gleichzeitig eine barrierefreie Interpretation ermöglicht. Auch bei großen Datensätzen muss die Übersichtlichkeit sowie eine interaktive Untersuchung einzelner Datenpunkte gewährleistet werden.

5.2.2 Abhängigkeiten

Die Heatmap stellt einen Bestandteil des Grafikpanels dar und war daher von mehreren zuvor entwickelten Programmkomponenten abhängig. Voraussetzung für die Erstellung war ein vorbereiteter und normalisierter Datensatz sowie die Ergebnisse der Clusteranalyse.

Die berechneten Distanzmatrizen, Cluster und Baumstrukturen liefern die Reihenfolge der Gene und Patienten, welche direkt die Anordnung der Zeilen und Spalten innerhalb der Heatmap bestimmen. Zusätzlich mussten die während der Datenintegration erzeugten Metadaten korrekt zugeordnet werden, damit diese später in der interaktiven Darstellung angezeigt werden konnten.

Da die Heatmap erst nach Fertigstellung dieser Komponenten vollständig getestet werden konnte, erfolgte die abschließende Optimierung erst nach der Implementierung der Clusteranalyse, Reihenfolgebestimmung und Farbpaletten.

5.2.3 Heatmap Erstellung

Die Funktion `generate_heatmap` erzeugt die Heatmap auf Grundlage des vorbereiteten Expressionsdatensatzes. Zunächst werden Gene und Patienten entsprechend der Ergebnisse der

Clusteranalyse sortiert, wodurch die Darstellung der ermittelten Gruppierungen entspricht.

Anschließend wird die Expressionsmatrix mithilfe von `melt` in das für `ggplot2` benötigte Long-Format überführt. Die Gennamen werden durch `make.unique` eindeutig gemacht und die Reihenfolge für die Darstellung angepasst.

Für die Visualisierung können verschiedene Farbpaletten ausgewählt werden. Die eigentliche Heatmap wird anschließend mit `ggplot2` erzeugt, wobei jeder Expressionswert durch ein farbiges Rechteck dargestellt wird. Zusätzlich wird eine Farbskala zur Interpretation der Expressionswerte eingebunden und die Achsendarstellung für größere Datensätze angepasst.

5.2.4 Einbau der Meta Daten

Die Funktion `extract_metadata_names` bereitet die Metadaten für die interaktive Darstellung auf. Hierzu werden alle mit `Meta_` gekennzeichneten Einträge erkannt und das Präfix entfernt. Dadurch können auch zukünftig hinzugefügte Metadaten automatisch verarbeitet werden, ohne dass eine erneute Anpassung notwendig ist.

5.2.5 Anordnung der Felddaten

Die Funktion `create_heatmap_field_data` erweitert die Expressionsdaten um die zugehörigen Metadaten. Hierzu werden die Metadaten den entsprechenden Patienten zugeordnet und als zusätzliche Spalten ergänzt. Dadurch entsteht die Grundlage für die Anzeige zusätzlicher Informationen innerhalb der interaktiven Plotly-Heatmap.

5.2.6 PDF Speicherung

Die Funktion `heatmap_pdf` ermöglicht den Export der Heatmap als PDF-Datei. Bei großen Datensätzen werden die Patienten automatisch auf mehrere Seiten verteilt, wodurch die Übersichtlichkeit und Lesbarkeit der Darstellung erhalten bleibt.

5.2.7 Konvertierung in Plotly

Die Funktion `generate_heatmap_plotly` wandelt die zuvor erstellte `ggplot2`-Heatmap in eine interaktive Plotly-Darstellung um. Dabei werden Gencode, Patientenbezeichnung und Expressionswert als Tooltip übernommen. Zusätzlich werden Zoom- und Verschiebefunktionen aktiviert, wodurch auch große Datensätze interaktiv untersucht werden können.

5.2.8 Grafikpanel

Die Heatmap wird gemeinsam mit den beiden Dendrogrammen dargestellt, da erst die Kombination aus Farbvisualisierung und Clusterstruktur eine aussagekräftige Interpretation der Genexpressionsdaten ermöglicht. Die Dendrogramme zeigen die Ähnlichkeiten zwischen Genen und Patienten, während die Heatmap deren Expressionswerte visualisiert. Für die Umsetzung wurden zunächst zwei Ansätze untersucht. Die Kombination über `patchwork` konnte nicht verwendet werden, da die GUI auf `plotly`-Elementen basiert und diese nicht direkt mit `patchwork`-Objekten kombinierbar sind.

Anschließend wurde die Erstellung über `plotly` Subplots getestet. Zwar konnten die einzelnen Darstellungen kombiniert werden, jedoch erfolgte das Zoomen jeweils nur für eine einzelne Grafik. Dadurch war keine synchrone Anpassung aller Komponenten möglich.

Schwierigkeiten Eine wesentliche Herausforderung stellte die allgemeine Zusammensetzung der einzelnen Visualisierungskomponenten zu einem gemeinsamen Grafikpanel dar. Dabei zeigte sich, dass die anfänglich gewählte Verwendung von `ggplot2` aufgrund der Nutzung von `plotly` in der GUI zu Einschränkungen führte. Zusätzlich war `patchwork` nicht mit `plotly`-Elementen kombinierbar. Durch fehlende Abstimmung zwischen den einzelnen Entwicklungsteams wurde diese Problematik erst spät erkannt.

Weitere Schwierigkeiten zeigten sich in den vereinbarten Deadlines. Da diese nicht eingehalten worden sind kam es zu zeitlichen Verschiebungen im weiteren Projektverlauf. Außerdem wurden kurz vorm Ende Fehler gefunden, diese kurzfristigen Änderungen haben zur großer Verwirrung geführt.

Aufgabe	Zuständig	Arbeitsaufwand
Recherche	Saliha	2 Stunden
Besprechungen	Saliha	19,5 Stunden
Bericht schreiben	Saliha	3 Stunden
Team Besprechungen	Saliha	8 Stunden
Heatmap Erstellung	Saliha	3,5 Stunden
Einbau der Meta Daten	Saliha	1 Stunden
Anordnung der Felddaten	Saliha	1 Stunden
PDF Speicherung	Saliha	1 Stunden
Konvertierung in Plotly	Saliha	2,5 Stunden
Grafikpanel Versionen	Saliha	4 Stunden
Testen	Saliha	15 Stunden

6 GUI

6.1 Arbeitsverteilung

Der Bereich der Graphical User Interfaces (GUIs) unterteilt sich in verschiedene Abschnitte. Vor allem sind die wichtigsten jeweils die Erstellung des Frontend-User-Interfaces, die Verbindung mit Back-end-Funktionen, PDF-Export, Sichern von Einstellungen, Einlesen von Datensätzen und Verschönerung des Layouts. Die folgende Tabelle zeigt die Arbeitsverteilung des Teams GUI.

Aufgabe	Zuständig	Arbeitsaufwand
Erste Visualisierung des Dashboards	Adrika	2 Stunden
Verbindung von allen Backend-Funktionen	Adrika	27 Stunden
Sichern von Einstellungen	Adrika	4 Stunden
Fehler Meldungen	Adrika	3 Stunden
Layout verschönern	Adrika	4.75 Stunden
Alle UI-Elemente implementieren	Adrika	15 Stunden
Bericht schreiben	Adrika	9.5 Stunden
Besprechungen	Adrika	25 Stunden
Erstellung von Folien	Adrika	3 Stunden

6.2 Verwendete Pakete

Die folgende Tabelle gibt eine Zusammenfassung aller verwendeten Pakete innerhalb von Shiny, die sehr wichtig sind, um ein Dashboard zu erstellen.

Paket	Aufgabe
shiny	R-Paket, mit dem man Web-Apps und Dashboards erstellen kann [6]
dipsaus	Add-on-Paket für Shiny, um praktische Funktionen und Verbesserungen hinzuzufügen [43]
shinydashboard	wichtig für die Erstellung eines einfachen Dashboard mit Header, Sidebar, und Body
shinyFeedback	wichtig für die Anzeige von Warnungen, oder generelle User-friendly Nachrichten
shinyjs	wichtig für die Verwendung von nützlichen JavaScript-Operationen [3]
bslib	wichtig für die Erstellung von custom bootstrapping-themes, was es einfacher macht Shiny-Apps zu stylen [38]
bsicons	Paket, wichtig für die Verwendung von Bootstrap Icons in Shiny [37]
shinyBS	bietet extra interaktive UI-Komponenten für Shiny-Apps wie Modal Pop-ups zum Beispiel [4]

6.3 Aufgaben

6.3.1 Entwicklung der Benutzeroberfläche

Zu Beginn des Projekts war es sehr wichtig, dass das Team GUI eine erste **Visualisierung des Dashboards** erstellte. Dieser Prototyp wurde mit Canva erstellt und lieferte einen ersten Einblick darin, wie das Dashboard aussehen sollte, damit es auf dieser Grundlage aufgebaut werden konnte. Dann wurde **R Shiny** verwendet, um diesen Prototyp zu einem echten Dashboard weiterzuentwickeln. Das war das Kernprogramm, um ein Dashboard für die Clusteranalyse zu entwickeln.

Mit dem **shinydashboard**-Paket war es möglich, ein leeres Dashboard mit Header, Sidebar und Body zu erstellen. Das war der grundlegende Aufbau des Projekts. Um die Benutzeroberfläche weiterzuentwickeln, war es am wichtigsten, verschiedene Tabs für unterschiedliche Nutzungen zu haben. Damit gab es eine Startseite, die die wertvollsten Informationen zu unseren Dashboards hatte, einen Datei-Hochladen-Tab, wobei der User eine CSV-Datei hochladen konnte und verschiedene Pathways auswählen durfte, einen Parameter-Auswahl-Tab zur Auswahl aller benötigten Parameter, um eine eindeutige und komplette Clusteranalyse durchzuführen, und am wichtigsten einen Visualisierungstabs, an welchem das Grafikpanel und die Dendrogramme erstellt wurden.

Zusätzlich gibt es bei den UI-Elementen von Shiny mehrere **Control Widgets**, um User-Eingabe zu kontrollieren und zu verarbeiten [1]. Als Erstes wurde die Pathways-Auswahl beim Dateihochladen erstellt. Weil es für einen User notwendig ist, mehrere Pathways auswählen zu können, wurde hier ein **SelectizeInput-Widget** verwendet, welches die Auswahl eines oder mehrerer Elemente aus einer Liste ermöglicht. Abbildung 2 zeigt ein Beispiel davon aus dem Dashboard. Daneben war bei der Parameter-Auswahl im Tab natürlich wichtig, dass der User die Möglichkeit hat, unterschiedliche Parameter auszuwählen, damit das Dashboard eine präzise und sinnvolle Clusteranalyse durchführen kann. Hierbei wurde eine Mischung aus **Radio-Buttons** und **Drop-down-Menü-Widgets** verwendet. Abbildung 5 zeigt, wie das Parameter-Auswahl-Tab am Ende aussieht. Bei der Clusterverfahrensauswahl, der Normalisierungsverfahrensauswahl und der Distanzmatrixauswahl wurden Drop-down-Menüs mit der Funktion **selectInput** angewendet. Bei der Farbpalettenauswahl war es aber sinnvoller, Radio-Buttons mit der Funktion **radioButtons** zu benutzen, weil es deutlich klarer und einfacher macht, Informationen über die in den Namen enthaltenen Farben umzusetzen.

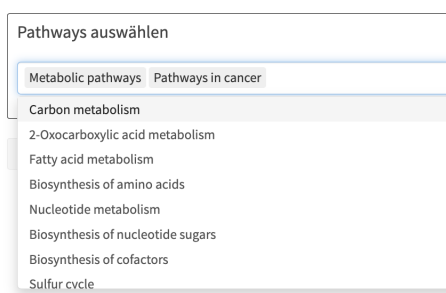


Abbildung 2: Beispiel der Pathway-Auswahl

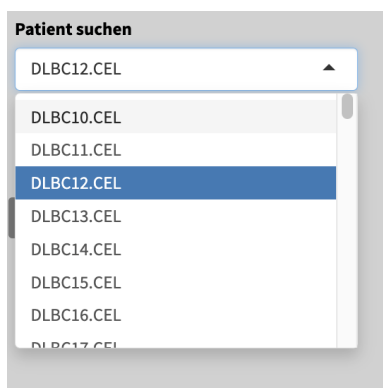


Abbildung 3: Beispiel der Patientensuche



Abbildung 4: Beispiel des Action Buttons

Parameter auswahl für Cluster Analyse

Abbildung 5: Beispiel der Parameterauswahl

Die gleichen Parameter wurden auch innerhalb der Sidebar beim Visualisierungs-Tab eingesetzt, damit es für den User einfacher ist, die Parameter direkt im gleichen Tab zu ändern. Jedoch war es wichtig, dass diese Parameter nur beim Visualisierungs-Tab angezeigt werden und nicht bei den anderen Tabs. Dafür wurde die **conditionalPanel**-Funktion eingesetzt, die ein Panel erstellt, das nur je nach dem Wert eines JavaScript-Ausdrucks angezeigt wird oder nicht. In diesem Fall musste es nur angezeigt werden, nachdem der User im Visualisierungs-Tab ankam. Für diese Funktion ist auch das shinyJS-Paket wichtig. Bei dem Tab wurde auch ein Suchfeld für Patienten eingebaut, wie auf der Abbildung 3 zu sehen ist. Das Ziel dieser Parameter ermöglicht es dem Arzt, auch mit der selectizeInput-Funktion nach einem Patienten innerhalb der Grafikpanels zu suchen. Dies wurde auch dynamisch eingesetzt, damit es sich für jeden Datensatz an die Patienten dynamisch anpasst.

Letztendlich wurden auch mehrere **Action-Buttons** eingebaut, welche einen einfachen Übergang zwischen den Tabs ermöglichten. Dies wurde auch mit der Funktion **actionButton** geführt. Ein Beispiel für so einen Action-Button ist der **“Run Cluster Analyse”** Button, der zum Visualisierungs-Tab führt und damit auch die Clusteranalyse durchführt. Dies wurde in Abbildung 4 gesehen.

6.3.2 Integration der Backend-Funktionen

Natürlich ist die Verbindung aller Backend-Funktionen ein sehr großer Teil des Dashboards, weil nämlich die Sachen zu einem funktionierenden Programm führen. Hier wurden nur Funktionen, die von anderen Teams geschrieben sind, aufgerufen und eingebunden. Erstens wurden die Funktionen der Cluster-Teams aufgerufen. Hier gab es die Normalisierungsfunktionen, das Distanzmatrixverfahren und das Clusteranalyseverfahren. Daneben gab es auch die Pre-Processing- und Analysefunktionen der Pathways. Danach kamen die Funktionen aus dem Visualisierungs-Team, wobei die zwei Dendrogramme und ein Grafikpanel aufgerufen wurden.

Viele von deren Funktionen wurden direkt in der Funktion **run_analysis** aufgerufen. Diese Funktion ist ein Kernprozess in der Shiny-Anwendung, weil sie mit dem „Run-Cluster“-Analyse-Action-Button verbunden ist und den kompletten Clusteranalyse-Prozess bis zum Endprodukt umfasst. Diese Funktion geht die hochgeladene CSV-Datei und die ausgewählten Pathways durch und wendet die Pre-process-Funktion an, die Daten vorbereitet. Danach wurden die Normalisierungsverfahren aufgerufen, wobei der User 7 Optionen für Verfahren hat. Ähnlicherweise wurden auch die Distanzmatrix-Optionen und Clusterverfahren-Optionen eingebaut. Alle diese Optionen wurden mit der **switch function** geschrieben. Diese Funktion wählt eine Auswahl

aus und wählt nur einen Wert aus einer Liste von Optionen. In diesem Fall, wenn der Arzt eine Option auswählt, wird diese von der Liste der Optionen herausgenommen und eingesetzt. Bei der Shiny-Anwendung wurden jedoch die UI-Namen und Elemente, die in Abbildung 5 gezeigt wurden, in backend-geeignete Werte übersetzt. Zum Beispiel wurden bei den Normalisierungsfunktionen für jede Funktion Zahlen eingesetzt, aber in der UI wurden sie durch ihre Namen erkannt. Deswegen war es wichtig, dass, wenn der User “**normalize_log_zscore**” auswählt, das Backend die **Nummer 1** einsetzt und damit die Users-Einstellung speichert.

Im Allgemeinen wurden aber auch **reactives** sehr oft verwendet, um die Einstellungen zu speichern. In Shiny sind solche reactives sehr wichtig, damit die Einstellungen direkt nach den Berechnungen automatisch gespeichert werden und damit auch, dass der Visualisierungs-Tab darauf später zugreifen kann. Im Wesentlichen, wenn sich irgendein Input ändert, wird der interne `run_analysis` Code durchlaufen und den aktualisierten Output in Echtzeit ausgeben. Das ist besonders wichtig, damit der User nicht lange Zeit auf einen Output warten muss. Einige Beispiele von den verwendeten Reactives sind: `cluster_result ← reactiveVal(NULL)` und `d_mat_result ← reactiveVal(NULL)`.

Schließlich sind für das Endprodukt des Prozesses die Visualisierungen durch Grafikpanel und Dendrogramme die allerwichtigsten. Hier wurden 3 Sub-Tabs innerhalb des großen Visualisierungstabs erstellt, wobei der User 3 Grafiken sehen kann. Einmal das gesamte Grafikpanel mit Heatmap und 2 Dendrogrammen und einmal das Patientendendrogramm und Genendendrogramm separat. Innerhalb der `run_analysis` Funktion wurden die vom User gewählten Clustering-Objekte eingelesen und in Visualisierungs-Objekte umgewandelt, je nach der Funktion der Visualisierungs-Teams. Diese Grafiken wurden anhand von Plotly aufgerufen. Plotly war sehr wichtig wegen der automatisch erstellten Zoom-Funktionen, die implementiert werden mussten. Diese Zoom-Funktion und am Ende die Patientensuche-Funktion waren sehr nützlich, um das Grafikpanel besser zu analysieren.

6.3.3 Verschönerung des Layouts

Als Team der GUI ist es auch sehr bedeutend, dass das Dashboard nicht nur praktisch funktioniert, sondern auch fesselnd ist und Benutzerfreundlichkeit für die User fördert. Im Allgemeinen wurden mehrere Besprechungen mit dem Farbenteam gehalten, um das Layout zu verschönern, damit es das Ziel des Dashboards erreicht. Für die Farben war es hauptsächlich wichtig, **Custom-CSS-Styles** zu verwenden, damit die Farben manipuliert und gesteuert werden konnten [2]. Hierbei wurden die Farben des Texts, der Action-Buttons, der Sidebar, des Headers, des Bodys und auch der Boxen für die Parameter selber manipuliert und gesteuert nach hexadezimalen Werten.

Außerdem, um die Benutzerfreundlichkeit weiter zu verbessern, wurden **Progress Bars** auch eingebaut, die die Progression der `run_analysis`-Funktion und des Dateihochladens zeigten. Das war implementiert, damit der User Bescheid wusste, dass der Analyseprozess angefangen hat, und nicht mehrmals auf den jeweiligen Action-Button drückte. Diese Fortschrittsbalken wurden mit der **withProgress-Funktion** und der **incProgress-Funktion** eingebaut, die Standardfunktionen in Shiny sind.

Um die Nutzerführung und das Verständnis zu erweitern, wurden ständig **Fehlermeldungen** und **Warnungen** implementiert. Mehrmals wurden die auch mit der `textbfshowNotification`-Funktion gebaut. Dies zeigt eine kleine Fehlermeldung in der Ecke, die den User auf eine falsche Eingabe, zum Beispiel, aufmerksam macht. Ein besonderes Beispiel ist die Parameterauswahl des Minkowski-Verfahrens. Es dürfen nur Zahlen größer als 1 und mindestens eine Zahl gegeben werden. Falls der User es vergisst, wird ein Fehler bei der `numericInput` geworfen. Dies wird in Abbildung 6 gezeigt. Zusätzlich wurden auch **Pop-up-Info-Boxen** erstellt, um den User

über wichtige Änderungen oder Informationen eindeutig und klar aufmerksam zu machen. Dies wurde durch die **modalDialog-Funktion** gemacht, so wie es in Abbildung 7 gezeigt wird. Damit hat der User die Möglichkeit, entweder weiterzugehen, abzubrechen oder diese Pop-up-Box nicht mehr zeigen zu lassen. Solche Elemente sind besonders wichtig, um die Usability eines Dashboards zu verbessern und eindeutiger zu machen. Endlich wurde auch mit dem **shinyFeedback**-Paket gearbeitet, wobei der `run_analysis`-Action-Button versteckt oder nicht drückbar ist, bis der User wertvolle Parameter eingibt, die wichtig sind, damit die Clusteranalyse richtig läuft. Zusätzlich wurden auch mithilfe des **bsicons**-Pakets viele Icons eingebaut, um das Layout weiter zu verschönern.

Parameter p eingeben

Bitte eine Zahl eingeben

Abbildung 6: Beispiel der Fehlermeldung

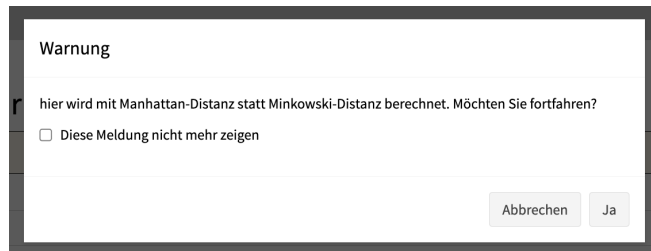


Abbildung 7: Beispiel der Pop-up-box

6.4 Herausforderungen

Die größte Herausforderung kam am meisten von den Anpassungen an alle Backend-Funktionen der verschiedenen Teams. Da wurden bis Ende viele Änderungen an den Visualisierungsfunktionen gemacht, und die mussten immer wieder angepasst und in Shiny geändert werden. Zweitens war eine der größten Herausforderungen nämlich das Debugging von vielen Abläufen in Shiny. Das war am Anfang eher problematisch, weil Shiny die Fehlermeldungen von Code nicht eindeutig zeigt, ohne Debugging-Codes selber zu erstellen. Zusätzlich war es am Anfang schwierig zu verstehen, wie Shiny überhaupt funktioniert, aber durch Videos und Demonstrationen der Shiny-Website wurde diese Herausforderung auch überwunden. Jedoch konnten durch Testen und schrittweise Anpassungen viele von diesen Problemen behoben werden.

6.5 Serverlogik

Aufgabe	Zuständig	Arbeitsaufwand
Vorbereitung des Datensatzes	Andreas	10 Stunden
Preset-Funktion	Andreas	5 Stunden
Cache	Andreas	7 Stunden
PDF-Export	Andreas	10 Stunden
Besprechungen	Andreas	20 Stunden

6.5.1 Verarbeitung des Datensatzes

Da nicht davon ausgegangen werden konnte, dass alle hochgeladenen Datensätze vollständig und ohne Fehler hochgeladen werden, wurde die Dateneingabe um eine automatische NA-Prüfung erweitert mit der fehlende Daten erkannt werden und die verhindert, dass fehlerhafte Werte in die Clusteranalyse übergeben werden mit denen sie nicht umgehen kann. Nach dem Upload wird berechnet, wie viele NA-Werte vorhanden sind und in welchen Zeilen und Spalten sie auftreten. Zeilen und Spalten, die vollständig aus fehlenden Werten bestehen, werden automatisch entfernt. Bei Zeilen oder Spalten mit mindestens 50 Prozent fehlenden Werten wird dagegen eine Entscheidung des Nutzers verlangt. Dabei können die betroffenen Zeilen oder Spalten entfernt oder beibehalten werden. Verbleibende NA-Werte werden anschließend durch Zeilenmittelwerte ersetzt. Die Informationen über die entfernte Zeilen, entfernte Spalten und ersetzte Werte werden in der GUI angezeigt. Die Implementierung dieser Funktionalität war eher schwierig, weil sich die Anforderungen während der Entwicklung mehrfach änderten bzw. man festgestellt hat das bestimmte Sachen nicht bedacht wurden oder auch schlecht kommuniziert wurde. Zu Beginn wurde die Möglichkeit der manuellen Behandlung jedes Wertes eingebaut, die in der Theorie sinnvoll erschien, später zeigte sich jedoch, dass die Lösung die Anwendung hauptsächlich komplizierter machten, ohne einen deutlichen Mehrwert zu bieten. Teilweise wurde viel Zeit in Ideen investiert, die später wieder vereinfacht oder vollständig entfernt wurden. Das automatische Entfernen einer Zeile mit vielen fehlenden Werten ist beispielsweise einfach umzusetzen, kann jedoch zum Verlust relevanter Gene führen. Daher wurde für kritische Fälle, oberhalb der 50 Prozent, eine Nutzerentscheidung eingeführt.

6.5.2 Preset-Funktion

Die Preset-Funktion wurde implementiert, um Einstellungen dauerhaft speichern und später einfacher wiederverwenden zu können. Ohne diese Funktion müssten die verschiedenen Parameter wie Clusterverfahren, die Normalisierung, die Distanzmethode, die Farbpalette, verschiedene numerische Parameter und die ausgewählten Pathways bei jeder Sitzung erneut eingestellt werden. Dies wäre besonders bei größeren Konfigurationen sehr zeitaufwendig und fehleranfällig. Die Einstellungen werden zunächst in einer Liste zusammengefasst und mithilfe des Pakets `jsonlite` als JSON-Datei gespeichert. Beim Laden wird die Datei wieder in eine Liste umgewandelt. JSON wurde gewählt, da das Format leicht gespeichert, eingelesen werden kann. Ein Preset enthält nur die Einstellungen der Analyse. Die größte Schwierigkeit bestand nicht im Speichern der JSON-Datei, sondern beim Laden des Presets. Es gibt viele Eingaben die konditional sind wie Minkowski mit dem P-Wert oder die Parameter alpha, beta und gamma ausschließlich für das Linkage-Verfahren benötigt werdender dann hinzugefügt werden mussten. Einige Einstellungen sind außerdem mehrfach in der Benutzeroberfläche vorhanden, beispielsweise auf der Parameterseite und zusätzlich in der Sidebar. Beim Laden eines Presets mussten deshalb nicht nur der Zustand der Parameter wählenSeite geupdatet werden sonder auch die zugehörigen Eingabefelder beim "VisualisierungReiter upgedatet werden, sonst hätte die GUI andere Werte angezeigt als für die

Berechnung verwendet worden wären. Die Pathways stellten einen weiteren Sonderfall dar, da die verfügbaren Werte zunächst aus der Datenbank geladen werden müssen.

6.5.3 Cache

Die Berechnung der Distanzmatrizen ist insbesondere bei größeren Datensätzen rechen- und zeitintensiv. Da sowohl eine Distanzmatrix für die Patienten als auch eine Distanzmatrix für die Gene benötigt wird, kann dieser Schritt einen großen Teil der Laufzeit ausmachen. Der Cache wurde deshalb implementiert, um bereits berechnete Matrizen wiederzuverwenden, wenn sich weder die Daten noch die entscheidenden Einstellungen geändert haben. Der Cache speichert die beiden Distanzmatrizen sowie einen Schlüssel, der die aktuelle Berechnungseinstellungen beinhaltet. Dieser Schlüssel berücksichtigt unter anderem die Datenmatrix, die Normalisierung, die Distanzmethode, die Pathway-Auswahl und wenn ausgewählt, den Minkowski-Parameter. Vor einer neuen Berechnung wird geprüft, ob der aktuelle Schlüssel mit dem gespeicherten Schlüssel übereinstimmt. Ist dies der Fall, können die vorhandenen Matrizen verwendet werden. Andernfalls werden sie neu berechnet und anschließend im Cache gespeichert. Die Schwierigkeit bestand hier darin zu entscheiden wann der Cache geleert werden muss und wann dieser noch gültig ist. Wird dies falsch entschieden können alte Distanzmatrizen mit neuen Daten oder Einstellungen kombiniert werden und dadurch ein fehlerhafter Report entsteht. Außerdem müssen hier wieder alle Parameter die in der GUI geändert werden können aufgenommen werden, da es hier wieder zwei Möglichkeiten gibt, die Parameter wählen-Seite und die Heatmap-Seite. Daher musste für jede Einstellung unterschieden werden, ob sie die Berechnung oder lediglich die Visualisierung beeinflusst oder die grundlegende Berechnung. Eine Änderung der Farbpalette benötigt beispielsweise keine neue Distanzmatrix. Eine andere Normalisierung, Distanzmethode oder Pathway-Auswahl verändert diese, auch der Datensatz selbst muss geprüft werden.

6.5.4 PDF-Export

Der PDF-Export wurde implementiert, um die Clusterergebnisse außerhalb der Anwendung anschauen und weitergeben zu können. Der PDF-Report enthält die verwendeten Parameter sowie das Patienten-Dendrogramm, das Gen-Dendrogramm und die Heatmap. Zu Beginn wurde versucht, den Bericht vollständig mit dem Paket `tinytex` zu erzeugen. Dieses funktionierte leider nicht und nach mehrmaligen versuch gab es immer einen Fehler aus. Anstatt die Ursache weiter zu suchen, wurde nach einer eigenen Lösung gesucht. Dabei werden die Parameterseite und die einzelnen Visualisierungen getrennt als PDF-Dateien erzeugt. Danach werden die Dateien sortiert und mit dem Paket `qpdf` zu einem gemeinsamen Bericht zusammengefügt. Dieser Ansatz war wesentlich aufwendiger als ursprünglich geplant, da die Parameterseite mit dem Base-R-Grafiksystem manuell gestaltet werden musste. Gleichzeitig mussten unterschiedliche Seitengrößen berücksichtigt werden. Besonders die Heatmap stellte bei großen Datensätzen mit mehreren hundert langen Gennamen ein Problem dar. Dafür wurden sowohl mehrseitige Darstellungen als auch eine einzelne große PDF-Seite getestet, auch die Reihenfolge musste stimmen bei der Zusammenkunft. Auch die Übertragung der Farbpalette und der korrekten Gennamen erforderte weitere Anpassungen. Rückblickend wäre es vermutlich einfacher gewesen, das Paket `tinytex` zum Laufen zu bringen und dessen Fehler zu beheben.

6.6 Farben

6.6.1 Kontrast- und Wahrnehmungsanforderungen

Die visuelle Aufbereitung der komplexen Darstellungen fordert fehlerfreie Interpretierbarkeit, Barrierefreiheit, visuelle Ausgeglichenheit und Neutralität. Dadurch sollen kognitive Verzerrungen durch ungleiche Signalwirkung vermieden und die Übersichtlichkeit beibehalten werden. Um dies zu erreichen, muss sichergestellt werden, dass die Extrempunkte einer Messreihe visuell mit derselben Intensität wahrgenommen werden. Ist dies nicht gewährleistet, kann das zu klinischen Fehlinterpretationen bezüglich der Relevanz bestimmter Labels führen. Die Berücksichtigung von Sehschwächen, insbesondere der Rot-Grün-Blindheit, wurde ebenfalls einbezogen, um einen höheren Grad der Barrierefreiheit zu erlangen [8].

Diese datenbezogenen Kontraste dürfen zudem nicht mit der Grafikoberfläche konkurrieren. Aus diesem Grund wurde für das gesamte Layout der GUI ein bewusst neutrales Farbschema aus Grau-, Weiß- und Beigetönen gewählt. Diese zurückhaltenden Farben minimieren die visuelle Ermüdung bei längerer Bildschirmnutzung und stellen sicher, dass der Fokus des Anwenders unverzerrt auf den biologischen Mustern im Grafikpanel liegt [40].

Da das Dashboard primär für die Betrachtung auf Bildschirmen konzipiert ist, wurden die Farben für den dabei verwendeten additiven RGB-Farbraum ausgelegt. Für den möglichen Farbdruck der exportierten PDF-Dateien, welcher im subtraktiven CMYK-Farbraum stattfindet, muss deshalb berücksichtigt werden, dass keine exakte Farbtreue zu erwarten ist [8].

6.6.2 Farbpaletten

Für die Wahl der Farbpaletten wurden sowohl theoretische als auch subjektive Farbwahrnehmungsaspekte in Betracht gezogen. Letztendlich wurden vier Farbpaletten ausgewählt, welche sowohl für die Heatmap als auch, in leicht abgewandelter Form, für das Dendrogramm verwendet wurden. Sie werden über die R-Pakete `RColorBrewer` [17] und `viridis` [14] angesteuert.

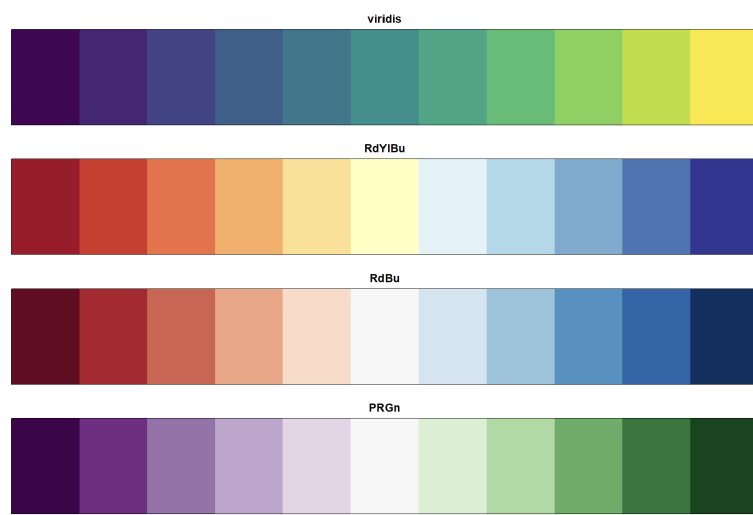


Abbildung 8: Übersicht der vier verwendeten Farbpaletten, erzeugt mit den R-Paketen `RColorBrewer` [17] und `viridis` [14].

Palette	Typ	Einsatzzweck und Begründung
RdYlBu	Divergierend	Bietet durch das helle, gelbe Zentrum einen neutralen Mittelpunkt für zweiseitige Abweichungen. Die Unterscheidung der äußeren Maxima mit Rot und Blau sind bewusst mit annähernd gleicher visueller Intensität gewählt und gut differenzierbar für Menschen mit einer Rot-Grün-Schwäche.
RdBu	Divergierend	Ein klassisches, in der Genexpressionsanalyse bewährtes Farbschema, das maximale Kontraste liefert. Da die Palette mit Rot, Weiß und Blau auf Grünanteile verzichtet, ist sie zudem für Personen mit Rot-Grün-Schwäche gut differenzierbar.
PRGn	Divergierend	Dient als kontrastreiche Alternative zu traditionellen Skalen. Da die Unterscheidung von Violett und Grün nicht auf der Rot-Grün-Achse, sondern zusätzlich über die Blau-Gelb-Wahrnehmung erfolgt, ist die Kombination für Personen mit der deutlich häufigeren Rot-Grün-Sehschwäche gut differenzierbar.
viridis	Sequentiell	Zeichnet sich durch einen gleichmäßigen, monoton steigenden Verlauf der Luminanz (Helligkeit) aus. Die visuelle Information bleibt auch bei Konvertierung in Graustufen oder Schwarz-Weiß-Druck vollständig erhalten.

Tabelle 3: Vergleich der implementierten Farbpaletten für die Heatmap-Generierung.

Die drei divergierenden Paletten RdYlBu, RdBu und PRGn sind dabei gezielt so konstruiert, dass ihre beiden äußeren Farbwerte eine annähernd gleiche Sättigung und Helligkeit aufweisen [17]. Dadurch wird vermieden, dass eine der beiden Richtungen einer Abweichung, etwa Über- oder Unterexpression eines Gens, allein aufgrund der Farbwahl visuell stärker gewichtet oder als bedeutsamer wahrgenommen wird als die andere.

Für die Heatmap wird von den Farbpaletten jeweils der vollständige, elfstufige Verlauf verwendet, bei der farblichen Kennzeichnung der Labels im Dendrogramm ergibt sich hingegen eine andere Anforderung. Während für die Heatmap eine sequentielle bzw. divergierende Ordnung kontinuierlicher Expressionswerte abgebildet werden muss, ist für die qualitative Kennzeichnung der Patienten eine unbekannte Anzahl von diskreten Gruppen, ohne natürliche Rangfolge nötig [17]. Um ein harmonisches Bild zu erzeugen orientiert sich die Farbgebung des Dendrogramms zwar an den Farbpaletten der Heatmap, verwendet jedoch nur neun der elf Farbabstufungen, da die mittleren hellen Töne oftmals nicht zuverlässig erkennbar und unterscheidbar sind.

Da es aber durchaus vorkommen kann, dass die Anzahl der zu kennzeichnenden Labels diese neun Farben übersteigt, existiert für diesen Fall ein sogenannter `default_colors`-Vektor. Er enthält 20 gut unterscheidbare Farben. Dabei wurde auf folgende Kriterien geachtet. Es wurden kaum stark leuchtende Neonfarben verwendet, da diese bei längerer Betrachtung zu visueller Ermüdung führen und unangemessen stark hervortreten würden [40]. Pastelltöne und schwer unterscheidbare Farben wurden ebenfalls ausgeschlossen, da sie auf dem hellen GUI-Hintergrund zu wenig Kontrast bieten würden. Zusätzlich wurde die Reihenfolge der Farben in der Liste so angepasst, dass stets ein deutlicher Farbabstand zwischen zwei aufeinanderfolgenden Labels besteht, um auch bei direkt benachbarter Vergabe eine gute Unterscheidbarkeit zu gewährleisten.

```
default_colors <- c(
  "#0000FF", "#FF0000", "#00FF00", "#D60072", "#B2DF8A",
  "#005300", "#FFD300", "#0096FF", "#9B4D00", "#00FFD2",
  "#A100FA", "#7B8100", "#960000", "#00646B", "#FDBF6F",
  "#FF6C00", "#540066", "#00A278", "#000094", "#FFC0CB"
)
```

Abbildung 9: Übersicht der 20 Farben des `default_colors`-Vektors.

6.6.3 Herausforderungen

Gut unterscheidbare Farbpaletten zu finden, die alle Anforderungen, vor allem die der Verzerrungsfreiheit erfüllen, aber auch die `default_colors` gut unterscheidbar auszuwählen, war nicht immer leicht. Die größten fachlichen Schwierigkeiten in diesem Projekt hat mir allerdings die farbliche Darstellung des Dendrogramms bereitet. Da die Linien dort sehr dünn auf weißem Hintergrund dargestellt werden, waren helle Farben aus den ausgewählten Farbpaletten häufig nicht gut erkennbar. Deshalb mussten die Abstufungen für das Dendrogramm entsprechend angepasst werden. Dieses Problem wurde leider auch erst vergleichsweise spät im Projektverlauf sichtbar, da die konzipierte Farbgebung erst spät ins Dendrogramm eingebaut wurde. Mit dem letztendlich erzielten Ergebnis bin ich zwar zufrieden, denke aber, dass sich eine noch feinere Abstimmung hätte erarbeiten lassen, wenn die getroffenen Absprachen von Beginn an konsequenter und fristgerecht eingehalten worden wären.

Aufgabe	Zuständig	Arbeitsaufwand
Recherche Farbdarstellungen	Johanna	7 Stunden
Testen verschiedener Farbtabellen	Johanna	2 Stunden
Datenfluss-Modell erstellt	Johanna	1 Stunden
Zusammenstellen der default-colors	Johanna	1,5 Stunden
Grundlagen Kapitel Biologie verfasst	Johanna	10 Stunden
Grundlagen Kapitel Medizin verfasst	Johanna	8 Stunden
Ausarbeitung Farben verfasst	Johanna	8 Stunden
Vorbereitung Präsentation	Johanna	5 Stunden
Kommunikation zwischen den Teams	Johanna	4 Stunden
Meetings mit Adrika	Johanna	3 Stunden
Absprachen mit Visualisierungsteam	Johanna	6 Stunden
Bericht Korrekturlesen	Johanna	2 Stunden
Gruppenmeetings	Johanna	19,5 Stunden

Literatur

- [1] Control widgets in shiny. <https://shiny.posit.co/r/getstarted/shiny-basics/lesson3/>.
- [2] Using custom css in your app. <https://shiny.posit.co/r/articles/build/css/>.
- [3] Dean Attali. *shinyjs: Easily Improve the User Experience of Your Shiny Apps in Seconds*, 2022. R package version 2.1.0.
- [4] Eric Bailey. *shinyBS: Extra Twitter Bootstrap Components for Shiny*, 2026. R package version 0.65.0.
- [5] Cancer Genome Atlas Research Network, John N. Weinstein, Eric A. Collisson, Gordon B. Mills, Kenna R. Shaw, Bradley A. Ozenberger, Kyle Ellrott, Ilya Shmulevich, Chris Sander, and Joshua M. Stuart. The Cancer Genome Atlas Pan-Cancer analysis project. *Nature Genetics*, 45(10):1113–1120, 2013.
- [6] Winston Chang, Joe Cheng, JJ Allaire, Carson Sievert, Barret Schloerke, Garrick Aden-Buie, Yihui Xie, Jeff Allen, Jonathan McPherson, Alan Dipert, and Barbara Borges. *shiny: Web Application Framework for R*, 2026. R package version 1.14.0.9000.
- [7] Ana Conesa, Pedro Madrigal, Sonia Tarazona, David Gomez-Cabrero, Alejandra Cervera, Andrew McPherson, Michał W. Szcześniak, Daniel J. Gaffney, Laura L. Elo, Xuegong Zhang, and Ali Mortazavi. A survey of best practices for rna-seq data analysis. *Genome Biology*, 17(1):13, 2016.
- [8] Fabio Crameri, Grace E. Shephard, and Philip J. Heron. The misuse of colour in science communication. *Nature Communications*, 11:5444, 2020.
- [9] F. Deng, C. H. Feng, N. Gao, and L. Zhang. Normalization and selecting non-differentially expressed genes improve machine learning modelling of cross-platform transcriptomic data. *Transactions on Artificial Intelligence*, 1(1):5, 2025.
- [10] Etienne Dubois, Antonio Núñez Galindo, Loïc Dayon, and Ornella Cominetti. Assessing normalization methods in mass spectrometry-based proteome profiling of clinical samples. *Biosystems*, 215-216:104661, 2022.
- [11] Blythe P. Durbin, Johanna S. Hardin, Douglas M. Hawkins, and David M. Rocke. A variance-stabilizing transformation for gene-expression microarray data. *Bioinformatics*, 18(Suppl. 1):105–110, 2002.
- [12] Dirk Eddelbuettel, Romain François, JJ Allaire, Kevin Ushey, Qiang Kou, Nathan Russell, Iñaki Ucar, Doug Bates, and John Chambers. *Rcpp: Seamless R and C++ Integration*, 2026. R package version 1.1.2.
- [13] Brian S. Everitt, Sabine Landau, Morven Leese, and Daniel Stahl. *Cluster Analysis*. Wiley, Chichester, 5 edition, 2011.
- [14] Simon Garnier, Noam Ross, Robert Rudis, Antônio P. Camargo, Marco Sciaini, and Cédric Scherer. *viridis(lite) – colorblind-friendly color maps for r*, 2024. R package.
- [15] Todd R. Golub, Donna K. Slonim, Pablo Tamayo, Christine Huard, Michelle Gaasenbeek, Jill P. Mesirov, Hilary Coller, Mignon L. Loh, James R. Downing, Michael A. Caligiuri, Clara D. Bloomfield, and Eric S. Lander. Molecular classification of cancer: class discovery and class prediction by gene expression monitoring. *Science*, 286(5439):531–537, 1999.
- [16] Zuguang Gu, Roland Eils, and Matthias Schlesner. Complex heatmaps reveal patterns and correlations in multidimensional genomic data. *Bioinformatics*, 32(18):2847–2849, 2016.

- [17] Mark Harrower and Cynthia A. Brewer. Colorbrewer.org: An online tool for selecting colour schemes for maps. *The Cartographic Journal*, 40(1):27–37, 2003.
- [18] Peter Horak, Christoph Heining, Sabrina Kreutzfeldt, Barbara Hutter, Annika Mock, Hamid Höck, et al. Clinical outcomes of molecular tumor boards: a systematic review. *JCO Precision Oncology*, 5:e21000495, 2021.
- [19] Iryna Ivchenko and Oleh Ivchenko. Hierarchical clustering taxonomy: From dendrograms to modern extensions. Intellectual Data Analysis Series, 2026.
- [20] Minoru Kanehisa, Yoko Sato, and Masayuki Kawashima. Kegg for taxonomy-based analysis of pathways and genomes. *Nucleic Acids Research*, 51(D1):D587–D592, 2023.
- [21] Sunil Kappal. Data Normalization Using Median and Median Absolute Deviation (MMAD) based Z-Score for Robust Predictions vs. Min-Max Normalization. *Great Britain Journal Press*, 2019.
- [22] Purvesh Khatri, Marina Sirota, and Atul J. Butte. Ten years of pathway analysis: Current approaches and outstanding challenges. *PLoS Computational Biology*, 8(2):e1002375, 2012.
- [23] Matthieu Komorowski, Dominic C. Marshall, Justin D. Saliccioli, and Yves Crutain. *Exploratory Data Analysis*, pages 185–203. Springer International Publishing, Cham, 2016.
- [24] Kimberly R. Kukurba and Stephen B. Montgomery. Rna sequencing and analysis. *Cold Spring Harbor Protocols*, 2015(11):pdb-top083808, 2015.
- [25] G. N. Lance and W. T. Williams. A general theory of classificatory sorting strategies: I. Hierarchical systems. *The Computer Journal*, 9(4):373–380, 1967.
- [26] Learn Statistics Easily. Dendrogramm, 2026. STATISTIK EINFACH LERNEN.
- [27] Tong Ihn Lee and Richard A. Young. Transcriptional regulation and its misregulation in disease. *Cell*, 152(6):1237–1251, 2013.
- [28] F. Mosele, J. Remon, J. Mateo, and et al. Recommendations for the use of next-generation sequencing (ngs) for patients with metastatic cancers: a report from the esmo precision medicine working group. *Annals of Oncology*, 31(11):1491–1505, 2020.
- [29] Shavira Narrandes and Wayne Xu. Gene expression detection assay for cancer clinical use. *Journal of Cancer*, 9(13):2249–2265, 2018.
- [30] Alicia Oshlack, Mark D. Robinson, and Matthew D. Young. From rna-seq reads to differential expression. *Genome Biology*, 11(12):220, 2010.
- [31] Julia Perera-Bel, Benjamin Hutter, Priya Balasubramanian, and et al. gmtb: A software tool for integrating tumor molecular profiles into clinical decision-making. *BMC Medical Informatics and Decision Making*, 18(1):72, 2018.
- [32] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2026.
- [33] Lior Rokach and Oded Maimon. *Clustering Methods*, pages 321–352. Springer US, Boston, MA, 2005.
- [34] Stefan Schmidt. Normalized median absolute deviation.
- [35] Margaret A. Shipp, Kenneth N. Ross, Pablo Tamayo, Andrew P. Weng, Jeffrey L. Kutok, Renato C. T. Aguiar, Michelle Gaasenbeek, Michael Angelo, Michael Reich, Geraldine S. Pinkus, Tapan S. Ray, Michael A. Koval, Kim W. Last, Andrew Norton, T. Andrew Lister,

- Jill Mesirov, Donna S. Neuberg, Eric S. Lander, Jon C. Aster, and Todd R. Golub. Diffuse large b-cell lymphoma outcome prediction by gene-expression profiling and supervised machine learning. *Nature Medicine*, 8(1):68–74, 2002.
- [36] Carson Sievert. *Interactive Web-Based Data Visualization with R, plotly, and shiny*. Chapman and Hall/CRC, 2020.
- [37] Carson Sievert. *bsicons: Easily Work with 'Bootstrap' Icons*, 2023. R package version 0.1.2.
- [38] Carson Sievert, Joe Cheng, and Garrick Aden-Buie. *bslib: Custom 'Bootstrap' 'Sass' Themes for 'shiny' and 'rmarkdown'*, 2026. R package version 0.11.0.
- [39] David Tamborero, Rodrigo Dienstmann, Mohamed Rachid, and et al. Support tools to guide clinical decision-making in precision oncology: the molecular tumor board. *NPJ Precision Oncology*, 4(1):22, 2020.
- [40] Edward R. Tufte. *The Visual Display of Quantitative Information*. Graphics Press, Cheshire, CT, 1983.
- [41] Robert A. van den Berg, Huub C. J. Hoefsloot, Johan A. Westerhuis, Age K. Smilde, and Mariët J. van der Werf. Centering, Scaling, and Transformations: Improving the Biological Information Content of Metabolomics Data. *BMC Genomics*, 7:142, 2006.
- [42] Bert Vogelstein, Nickolas Papadopoulos, Victor E. Velculescu, Shibin Zhou, Luis A. Diaz, and Kenneth W. Kinzler. Cancer genome landscapes. *Science*, 339(6127):1546–1558, 2013.
- [43] Zhengjia Wang. *dipsaus: A Dipping Sauce for Data Analysis and Visualizations*, 2026. R package version 0.3.4, <https://dipterix.org/dipsaus/>.
- [44] Zhong Wang, Mark Gerstein, and Michael Snyder. Rna-seq: a revolutionary tool for transcriptomics. *Nature Reviews Genetics*, 10(1):57–63, 2009.
- [45] Hadley Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2016.